

数据传输指令

8086指令系统

从功能上包括六大类：



数据传送

算术运算

逻辑运算和移位

串操作

程序控制

处理器控制

掌握：

- 指令码的含义
- 指令对操作数的要求
- 指令的对标志位的影响
- 指令的功能

数据传送类指令

1. 通用数据传送指令
2. 输入输出指令
3. 地址传送指令
4. 标志传送指令

除标志传送指令外，其它指令的执行对标志位不产生影响

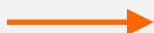
一、通用数据传送指令

通用数据传送

- 一般数据传送指令
- 堆栈操作指令
- 交换指令
- 查表转换指令
- 字位扩展指令

该类所有指令的执行均不影响标志位

1. 一般数据传送指令

- 一般数据传送指令 **MOV**
- 格式：
 - **MOV dest, src**
- 操作：
 - **src**  **dest**
- 例：
 - **MOV AL, BL**

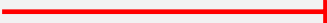
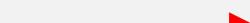
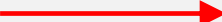
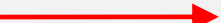
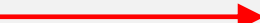
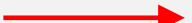
一般数据传送指令

■ 注意点:

- 两操作数字长必须相同;
- 两操作数不允许同时为存储器操作数;
- 两操作数不允许同时为段寄存器;
- 在源操作数是立即数时, 目标操作数不能是段寄存器;
- IP和CS不作为目标操作数, FLAGS一般也不作为操作数在指令中出现。

例：

■ 判断下列指令的正确性：

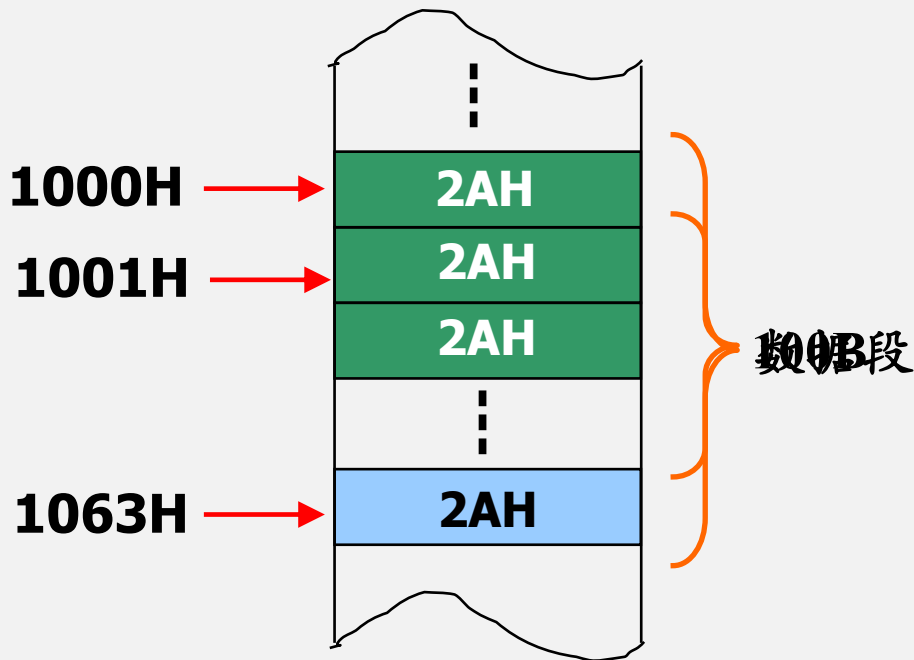
- **MOV AL, BX**  **X** 两操作数字长不相等
- **MOV AX, [SI]05H**  **✓** 源操作数为相对寻址
- **MOV [BX][BP], BX**  **X** 目标操作数寻址方式错误
- **MOV DS, 1000H**  **X** 不能用立即寻址方式为段寄存器赋值
- **MOV DX, 09H**  **✓**
- **MOV [1200], [SI]**  **X** 两操作数不能同时为存储器操作数

一般数据传送指令应用例

- 将(*)的ASCII码2AH送入内存数据段1000H开始的100个单元中。

- 题目分析:

- 确定首地址
- 确定数据长度
- 写一次数据
- 修改单元地址
- 修改长度值
- 判断写完否?
- 未完继续写入，否则结束



一般数据传送指令应用例

程序段：

MOV DI, 1000H

MOV CX, 64H

MOV AL, 2AH

AGAIN: MOV [DI], AL

INC DI

; DI+1

DEC CX

; CX-1

JNZ AGAIN

; CX≠0则继续

HLT

指令助记符

操作数

注释

上段程序在代码段中的存放形式

- 設CS=109EH, IP=0100H, 則各条指令在代码段中的存放地址如下:

CS :	IP	机器指令	汇编指令
109E:	0100	B80010	MOV DI, 1000H
109E:	0103		MOV CX, 64H
109E:	0105		MOV AL, 2AH
109E:	0107		MOV [DI], AL
109E:	0109		INC DI
109E:	010A		DEC CX
109E:	010B		JNZ 0107H
109E:	010D		HLT

数据段中的分布

送上2AH后数据段中相应存储单元的内容改变如下：

DS: 1000	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1010	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1020	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1030	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1040	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1050	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1060	2A	2A	2A	2A	00	00	00	00	00	00	00	00	00	00	00	00	00



偏移地址[DI]

2. 堆栈操作指令

- 堆栈操作的原则

- 先进后出
- 以字为单位

- 堆栈操作指令：

- 压栈指令

- 格式: **PUSH OPRD**

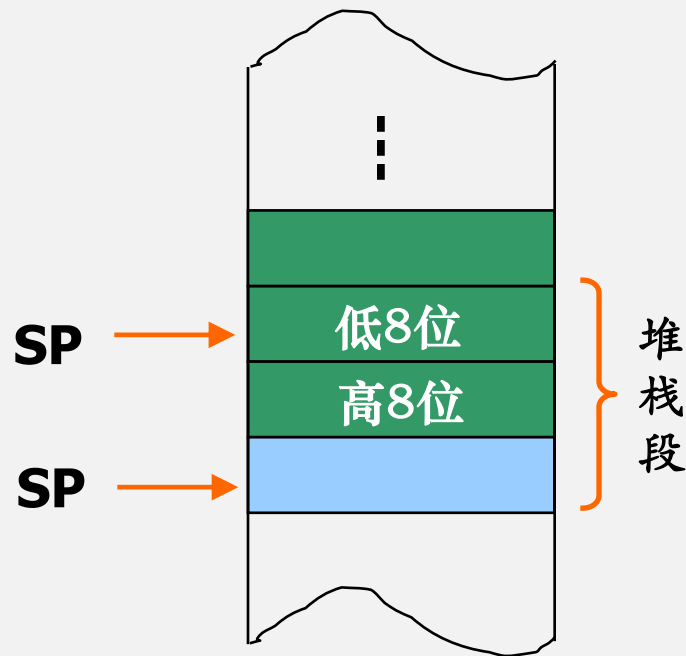
16位寄存器或
存储器两单元

- 出栈指令

- 格式: **POP OPRD**

压栈指令 PUSH

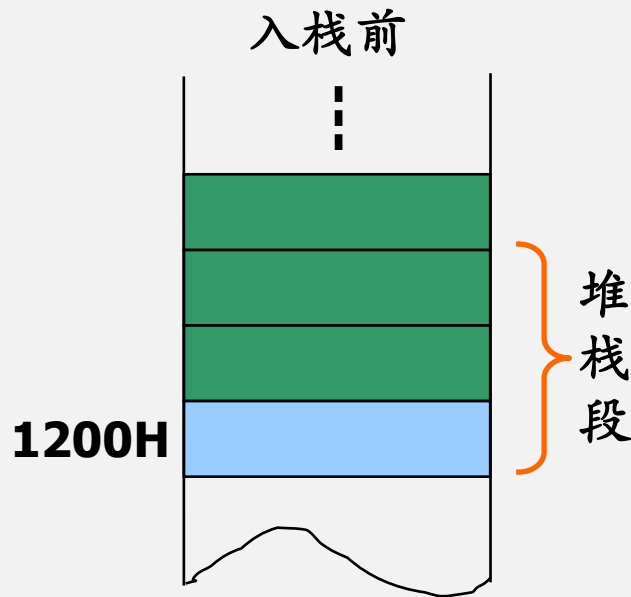
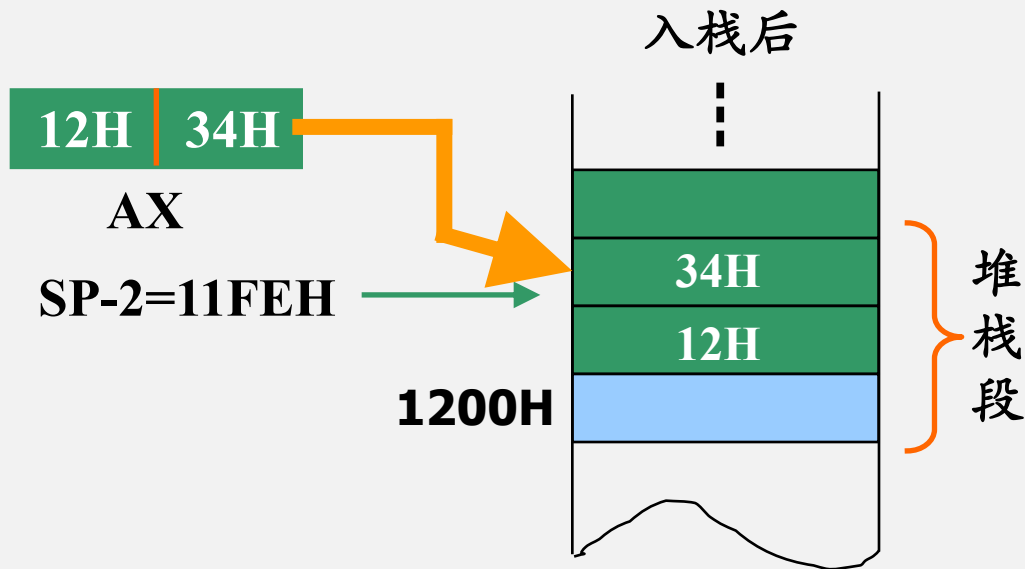
- 指令执行过程:
 - $SP - 2 \rightarrow SP$
 - 操作数高字节 $\rightarrow SP+1$
 - 操作数低字节 $\rightarrow SP$



压栈指令的操作

设 **AX=1234H**, **SP=1200H**

执行 **PUSH AX** 指令后堆栈区的状态:



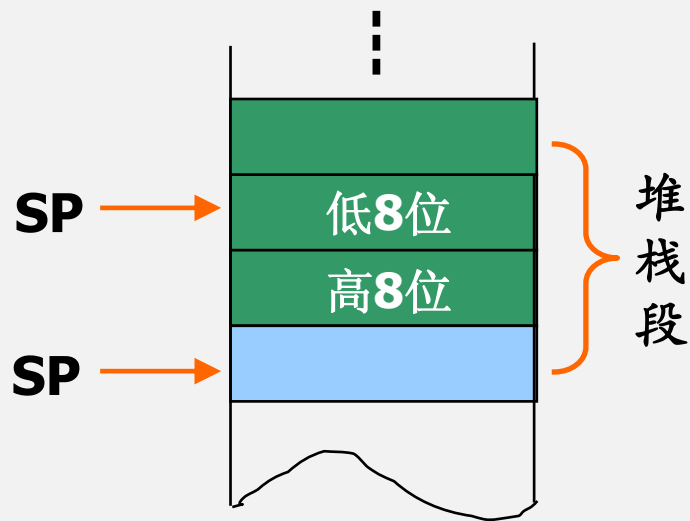
出栈指令POP

■ 指令执行过程:

SP → 操作数低字节

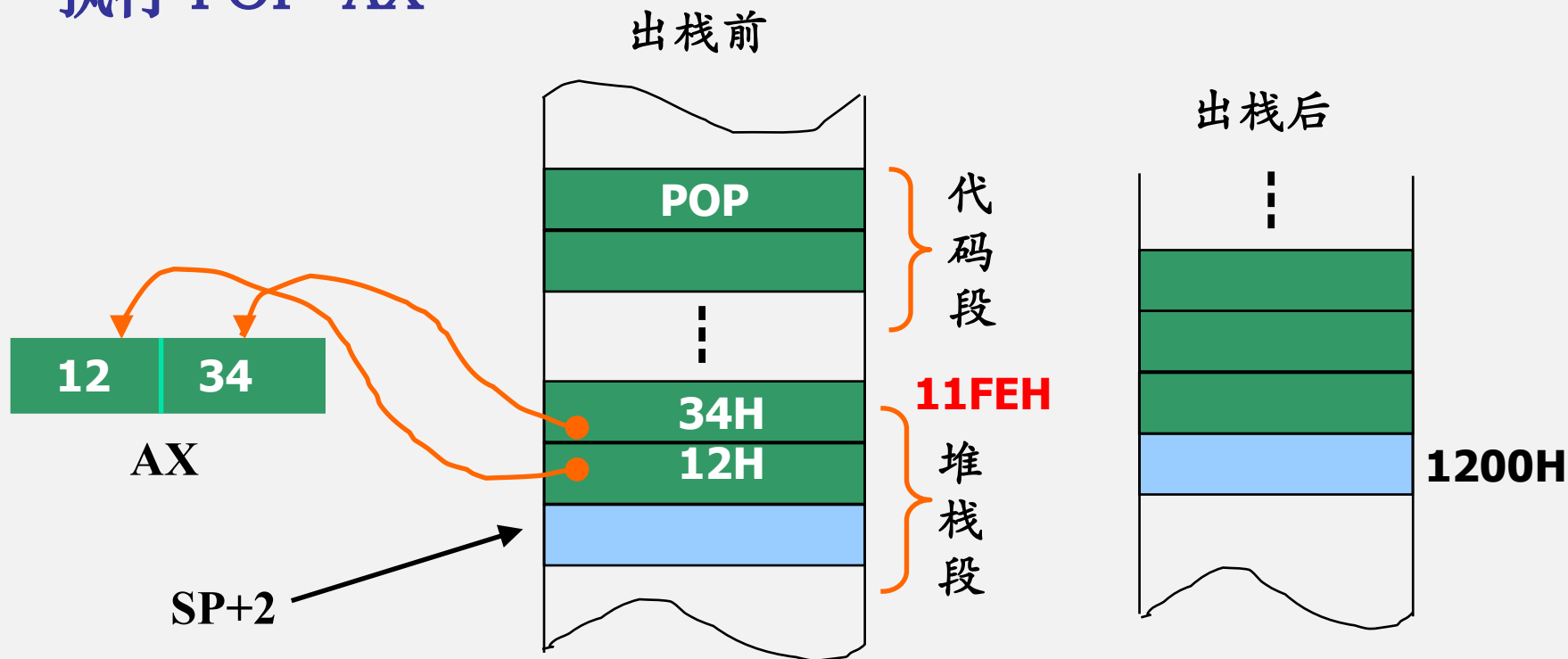
SP+1 → 操作数高字节

SP ← SP+2



出栈指令的操作

执行 POP AX

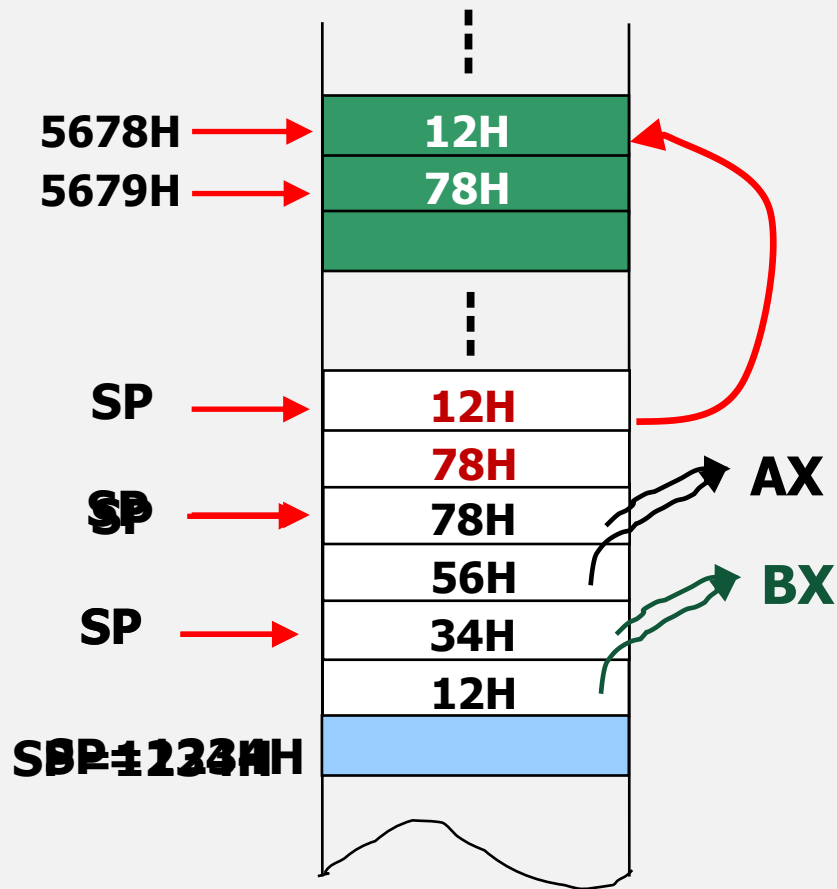


堆栈操作指令说明

- 指令的操作数必须是16位；
- 操作数可以是寄存器或存储器两单元，但不能是立即数；
- 不能从栈顶弹出一个字给CS；
- PUSH和POP指令在程序中一般成对出现；
- PUSH指令的操作方向是从高地址向低地址，而POP指令的操作正好相反。

堆栈操作指令例

- **MOV AX, 1234H**
- **MOV SP, AX**
- **MOV BX, 5678H**
- **MOV [BX], AH**
- **MOV [BX+1], BL**
- **PUSH AX**
- **PUSH BX**
- **PUSH WORD PTR[BX]**
- **POP WORD PTR[BX]**
- **POP AX**
- **POP BX**



使AX和BX的内容互换

3. 交换指令

- 格式:

- **XCHG REG, MEM/REG**

- 注:

- 两操作数必须有一个是寄存器操作数
- 不允许使用段寄存器。

- 例:

- **XCHG AX, BX**
- **XCHG [2000], CL**

4. 查表指令

- 格式:

- XLAT

- 说明:

- 用BX的内容代表表格首地址, AL内容为表内位移量, $BX+AL$ 得到要查找元素的偏移地址

- 操作:

- 将 $BX+AL$ 所指单元的内容送AL

5. 字位扩展指令

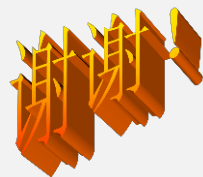
- 将符号数的符号位扩展到高位;
- 指令为零操作数指令，采用隐含寻址，隐含的操作数为 **AX** 及 **AX, DX**
- 无符号数的扩展规则为在高位补0

字节到字的扩展指令

- 格式：
 - CBW
- 操作：
 - 将AL内容扩展到AX
- 规则：
 - 若最高位=1，则执行后AH=FFH
 - 若最高位=0，则执行后AH=00H

字到双字的扩展指令

- 格式：
 - **CWD**
- 操作：
 - 将AX内容扩展到DX AX
- 规则：
 - 若最高位=1，则执行后DX=FFFFH
 - 若最高位=0，则执行后DX=0000H



二、输入输出指令

输入输出指令

掌握：

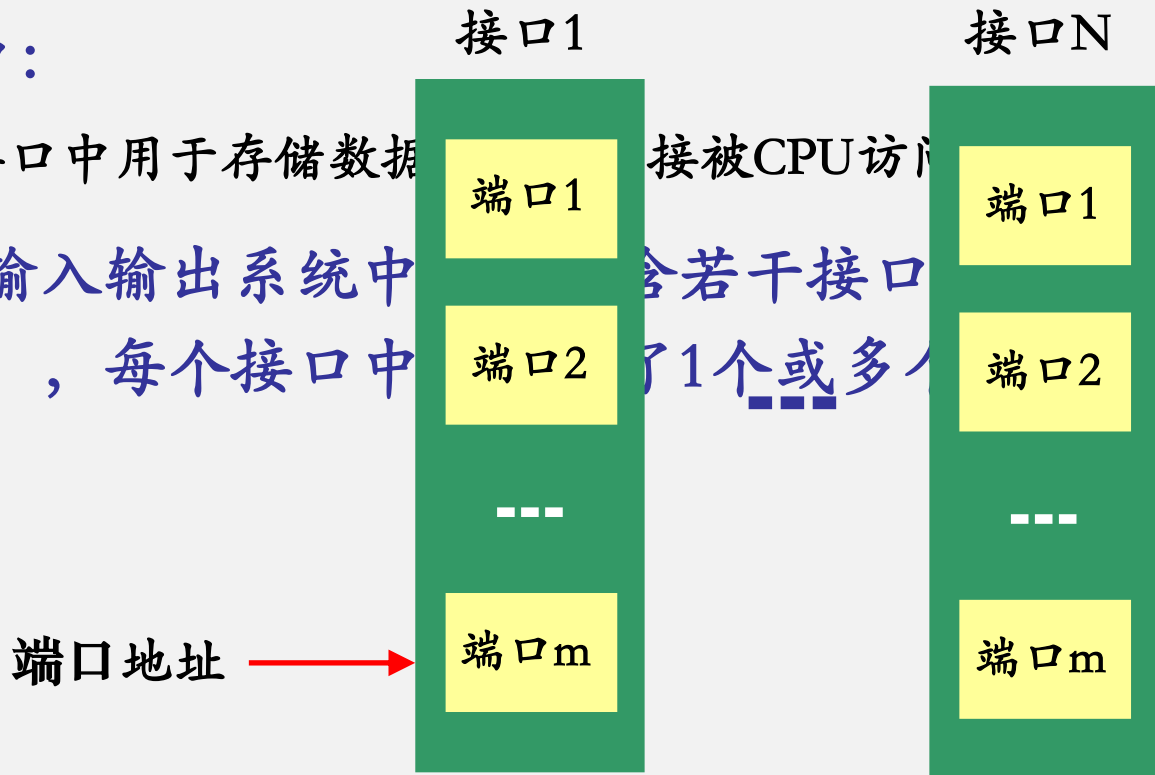
- 指令的格式及操作
- 指令的两种寻址方式
- 指令对操作数的要求

关于I/O端口

- I/O端口：

- I/O接口中用于存储数据的寄存器，每个寄存器都被CPU访问。

- 计算机输入输出系统中包含若干接口，每个接口都包含一个或多个寄存器（芯片），每个接口中



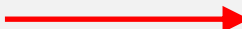
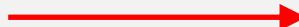
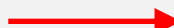
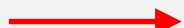
输入输出指令

- 专门面向I/O端口操作的指令
- 端口地址在指令中的表示方式 —— 寻址方式
- 指令功能：
 - 从端口地址读入数据到累加器/将累加器的值输出到端口中
- 指令格式：
 - 输入指令：IN acc, PORT 
 - 输出指令：OUT PORT, acc 

指令寻址方式

- 根据端口地址码的长度，指令具有两种不同的端口地址表现形式。
- 直接寻址
 - 端口地址为8位时，指令中直接给出8位端口地址；
 - 寻址256个端口。
- 间接寻址
 - 端口地址为16位时，指令中的端口地址必须由DX指定；
 - 可寻址64K个端口。

I/O指令例

- **IN AX, 80H**  从80H端口读入16bit数据到AX
- **MOV DX, 2400H**
- **IN AL, DX**  从2400H端口读入8bit数据到AL
- **OUT 35H , AX**  将AX的值写入到35H端口中
- ***OUT AX, 35H***  × 格式错误

小结

■ 例：

- ① MOV SI, 100
- ② MOV DX, 03F8H
- ③ IN AL, DX
- ④ 如果AL的最高位=0，则转向③，否则继续下一步
- ⑤ MOV AX, [SI]
- ⑥ OUT 58H, AX

程序
功能？



三、地址传送指令

地址传送指令

- 取偏移地址指令LEA → 取近地址指针
 - *LDS指令
 - *LES指令
- } → 取远地址指针

1. LEA指令

■ 操作：

- 将变量的16位偏移地址写入到目标寄存器

- 当程序中用符号表示内存单元的地址时，须使用该指令。

号地址。属于
存储器操作数

■ 格式：

- LEA REG, MEM

必须是存
储操作数

■ 指令要求：

- 源操作数必须是一个存储器操作数，目标操作数通常是间址寄存器。

LEA指令与MOV指令执行结果对比

■ 执行指令：

- MOV AL, i
- 结果：AL=4

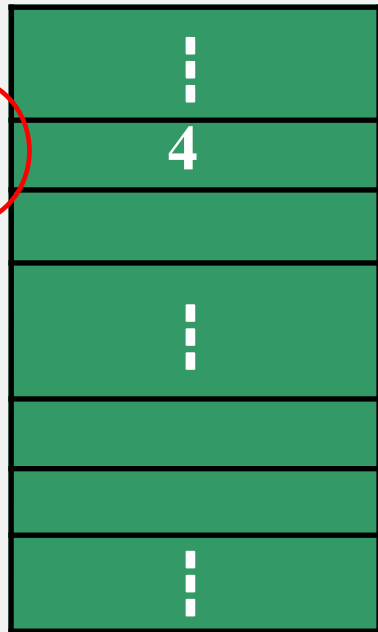
MOV指令读取指定内存单元的内容
源操作数为直接寻址方式

■ 指令LEA指令：

- LEA BX, i
- 结果：BX=i

LEA指令读取指定内存单元的偏移地址（获得i值本身）

变量



LEA指令与MOV指令执行结果对比

■ 比较下列指令：

■ `MOV SI, DATA1`

符号
地址

■ 执行结果：**SI=1234H**

执行结果：**SI=DATA1**

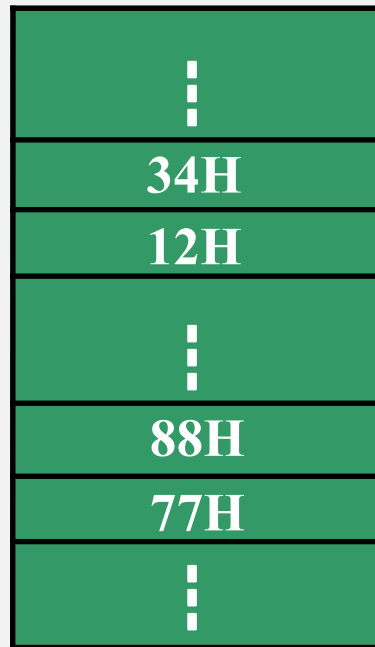
■ `MOV BX, 1100H`

■ `MOV AX, [BX]` → **AX=7788H**

■ `LEA BX, [BX]` → **BX=1100H**

DATA1

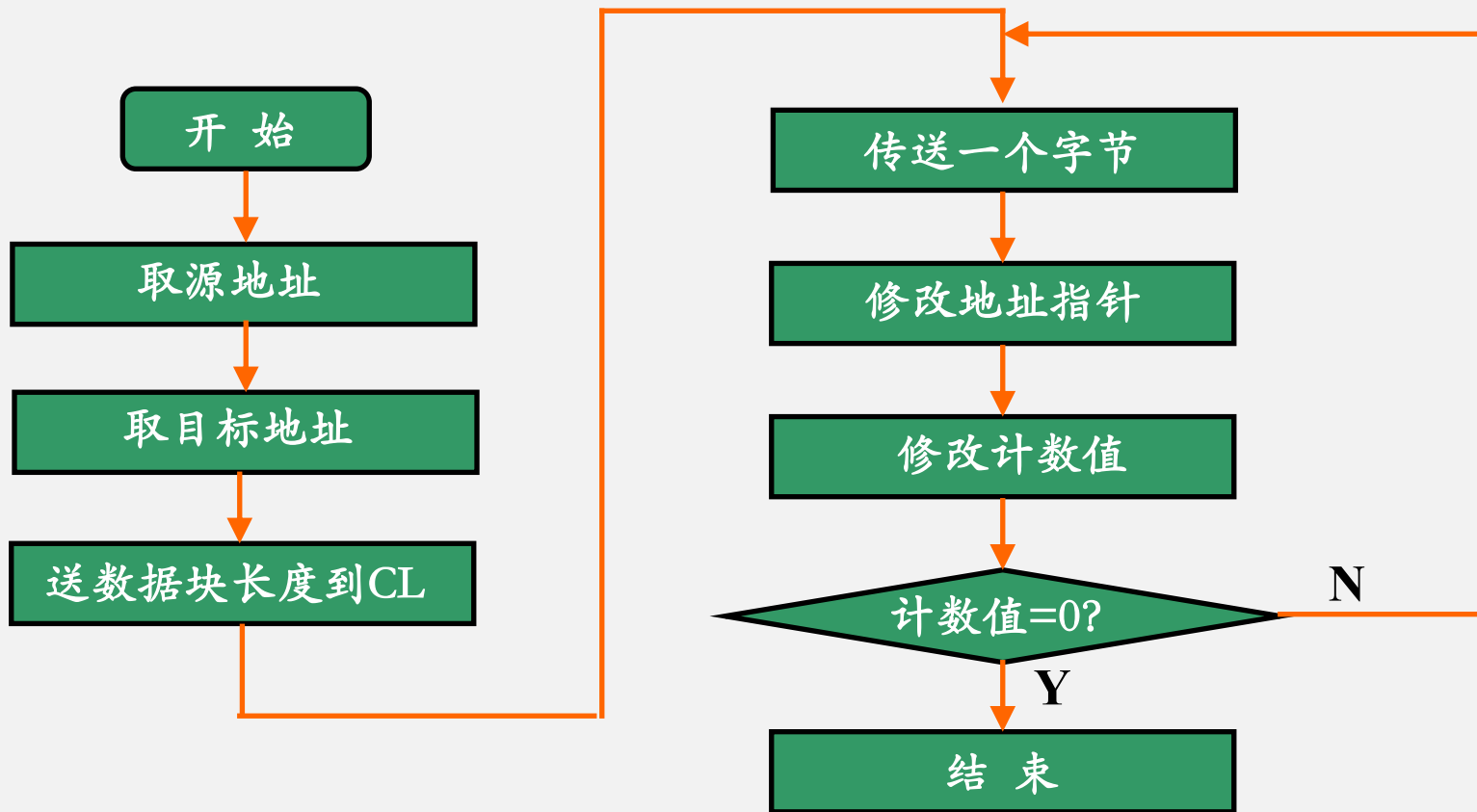
1100H



LEA指令在程序中的应用

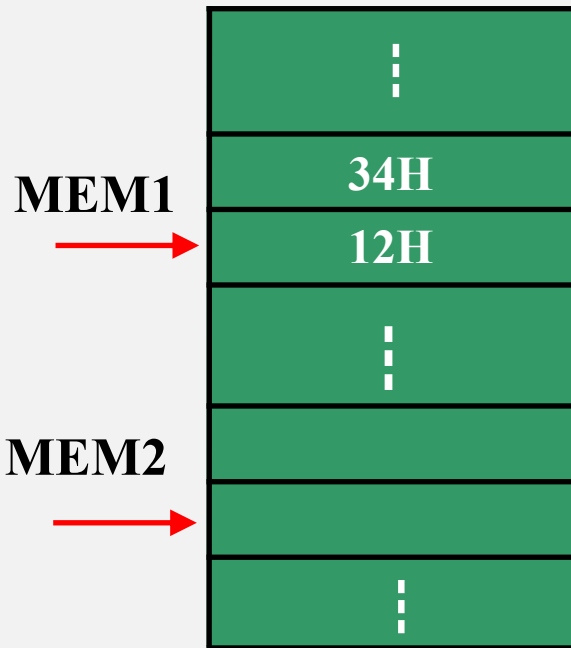
- 将数据段中首地址为MEM1 的50个字节的数据传送到同一逻辑段首地址为MEM2的区域存放。编写相应的程序段。

LEA指令在程序中的应用



LEA指令在程序中的应用

```
LEA SI, MEM1
LEA DI, MEM2
MOV CL, 50
NEXT: MOV AL, [SI]
      MOV [DI], AL
      INC SI
      INC DI
      DEC CL
      JNZ NEXT
      HLT
```



CL $\neq$ 0 则转 NEXT

2. LDS、LES指令

- LDS和LES均用于将一个32位的远地址指针写入到目标寄存器。
- LDS (Load pointer using DS) 的一般格式：
 - LDS 通用寄存器, 存储器操作数  将源操作数的偏移地址送目标寄存器, 将源操作数的段地址送DS
- LES (Load pointer using ES) 的一般格式：
 - LES 通用寄存器, 存储器操作数  将源操作数的偏移地址送目标寄存器, 将源操作数的段地址送ES

四、标志传送指令

标志传送指令

LAHF (Load AH from Flags)

SAHF (Store AH into Flags)

} 隐含操作数AH

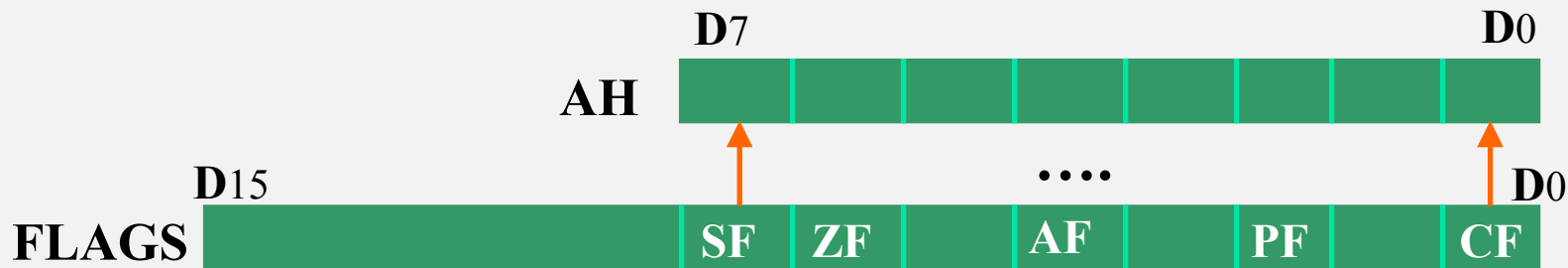
PUSHF (Push flags onto stack)

POPF (Pop flags off stack)

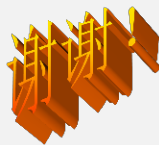
} 隐含操作数FLAGS

LAHF , SAHF

- 指令格式：
 - LAHF
- 操作：将FLAGS的低8位装入AH



SAHF执行与**LAHF**相反的操作



算术运算类指令

算术运算类指令

- 加法运算指令
- 减法运算指令
- 乘法指令
- 除法指令

算术运算指令的执行大多对状态标志位会产生影响

一、加法运算指令

加法指令

- 普通加法指令ADD
- 带进位位的加法指令ADC
- 加1指令INC

加法指令对操作数的要求与MOV指令相同

1. ADD指令

- 格式:

- **ADD OPRD1, OPRD2**

- 操作:

- **OPRD1+OPRD2 $\longrightarrow$ OPRD1**

ADD指令的执行对全部6个状态标志位都产生影响

ADD指令例

MOV AL, 78H

ADD AL, 99H

指令执行后6个状态标志位的状态

ADD指令例

$$\begin{array}{r} 01111000 \\ + 10011001 \\ \hline \boxed{1} 00010001 \end{array}$$

标志位状态： CF= **1** SF= **0**

AF= **1** ZF= **0**

PF= **1** OF= **0**

2. ADC指令

- 指令格式、对操作数的要求、对标志位的影响与ADD指令完全一样
- 指令的操作：
 - $\text{OPRD1} + \text{OPRD2} + \text{CF} \longrightarrow \text{OPRD1}$

ADC指令多用于多字节数相加，使用前要先将CF清零

3. INC指令

- 格式:

- INC OPRD

不能是段寄存器
不能是立即数

- 操作:

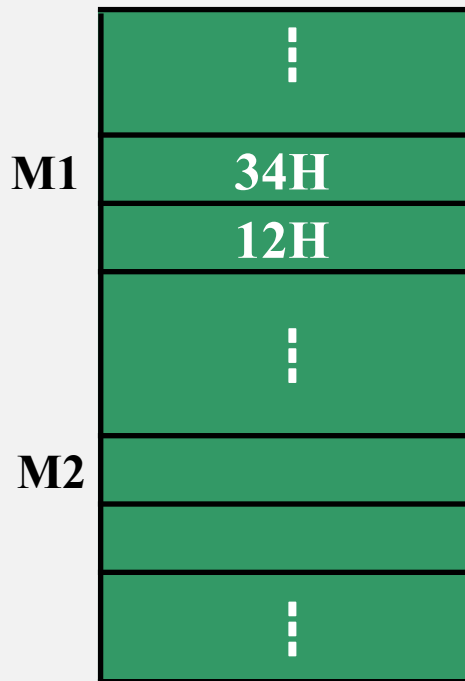
- $\text{OPRD} + 1 \longrightarrow \text{OPRD}$

常用于在程序中修改地址指针

加法指令例：

求内存数据段中M1为首和M2为首的两个20字节数之和，并将结果写入M2为首的区域中。

```
LEA SI, M1
LEA DI, M2
MOV CX, 20
CLC      → 使CF=0
NEXT: MOV AL, [SI]
      ADC [DI], AL
      INC SI
      INC DI
      DEC CX
      JNZ NEXT
      HLT
```



二、减法运算指令

减法指令

- 普通减法指令SUB
- 考虑借位的减法指令SBB
- 减1指令DEC
- 比较指令CMP
- 求补指令NEG

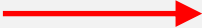
减法指令对操作数的要求与对应的加法指令相同

1. SUB指令

- 格式:

- SUB OPRD1, OPRD2

- 操作:

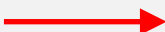
- **OPRD1- OPRD2  OPRD1**

- **对标志位的影响与ADD指令同**

2. SBB指令

- 指令格式、对操作数的要求、对标志位的影响与SUB指令完全一样
- 指令的操作：
 - $\text{OPRD1} - \text{OPRD2} - \text{CF} \longrightarrow \text{OPRD1}$

3. DEC指令

- 格式：
 - **DEC OPRD**
- 操作：
 - **OPRD - 1  OPRD**

指令对操作数的要求与INC相同

指令常用于在程序中修改计数值

应用程序例

```
        MOV BL, 2
NEXT1 : MOV CX, 0FFFFH
NEXT2:  DEC CX
        JNZ NEXT2      ; ZF=0转NEXT2
        DEC BL
        JNZ NEXT1      ; ZF=0转NEXT1
        HLT            ; 暂停执行
```

程序功能：
延时（定时）

4. NEG指令

- 格式:

- **NEG OPRD**

8/16位寄存器或
存储器操作数

- 操作:

- **0 - OPRD $\longrightarrow$ OPRD**

对一个负数取补码就相当于用零减去此数

NEG指令

■ 说明：

- 执行NEG指令后，一般情况下都会使CF为1，除非给定的操作数为零才会使CF为0；
- 当指定的操作数的值为80H(-128)或为8000H(-32768)，则执行NEG指令后，结果不变，但OF置1，其它情况下OF均置0。

用0减去操作数，可以得到负数的绝对值

5. CMP指令

- 格式：
 - **CMP OPRD1, OPRD2**
- 操作：
 - **OPRD1- OPRD2**

指令执行的结果不影响目标操作数，仅影响标志位！

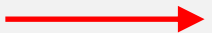
CMP指令

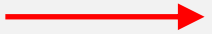
- 用途：
 - 用于比较两个数的大小，可作为条件转移指令转移的条件
- 指令对操作数的要求及对标志位的影响与SUB指令相同

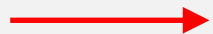
CMP指令

- 两个无符号数的比较:

- **CMP AX, BX**

- 若 $AX \geq BX$  **CF=0**

- 若 $AX < BX$  **CF=1**

- 若 $AX=BX$  **CF=0, ZF=1**

CMP指令

- 两个带符号数的比较
 - **CMP AX, BX**

两个数的大小由OF和SF共同决定

OF和SF状态相同 $AX \geq BX$

OF和SF状态不同 $AX < BX$

CMP指令例

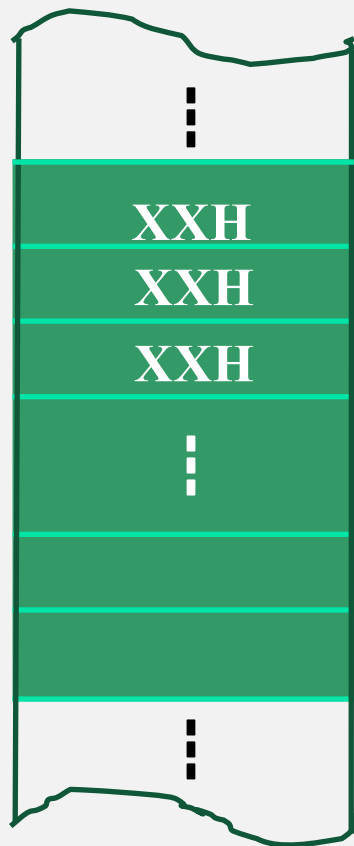
1.	LEA BX, MAX	9.	GOON: DEC CL
2.	LEA SI, BUF	10.	JNZ NEXT
3.	MOV CL, 20	11.	MOV [BX], AL
4.	MOV AL, [SI]	12.	HLT
5.	NEXT: INC SI		
6.	CMP AL, [SI]		
7.	JNC GOON ; CF=0转移		
8.	XCHG [SI], AL		

程序功能

在20个数中找最大的数，并将其存放在MAX单元中。

└───────────▶ MAX

BUF



谢谢

三、乘除运算指令

1. 乘法指令

{ 无符号的乘法指令MUL
带符号的乘法指令IMUL

■ 注意点:

- 乘法指令采用隐含寻址，隐含的是存放被乘数的累加器AL或AX及存放结果的AX，DX；

无符号数乘法指令

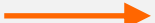
- 格式:

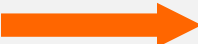
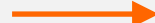
- **MUL OPRD**



不能是立即数

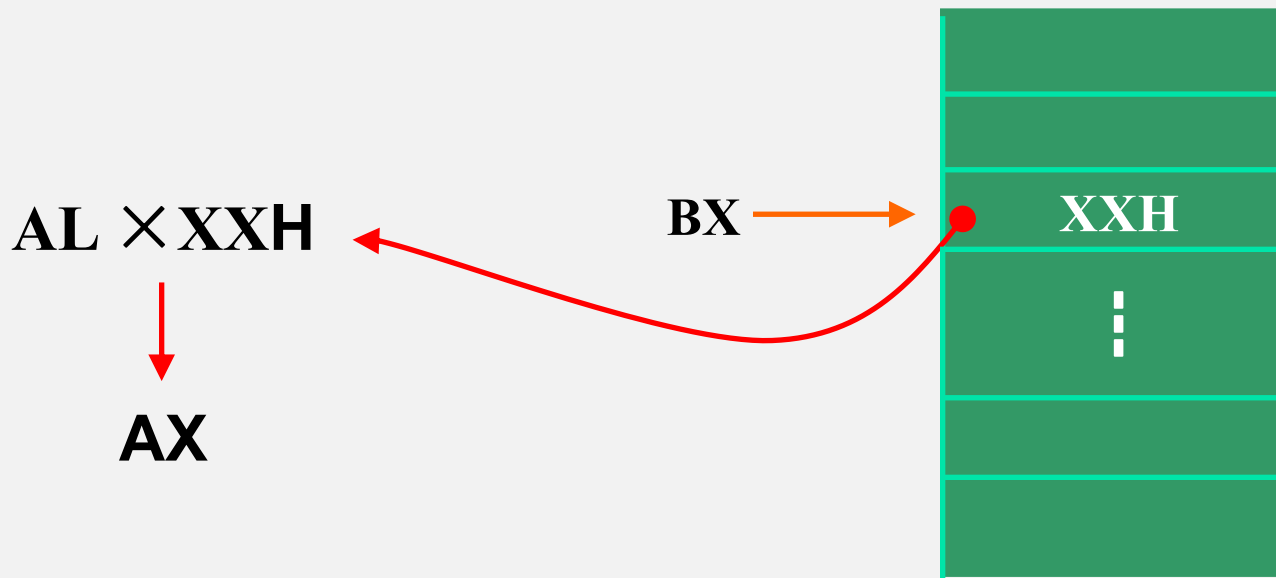
- 操作:

- OPRD为字节数  **AL × OPRD**  **AX**

- OPRD为16位数  **AX × OPRD**  **DXAX**

无符号数乘法指令例

- MUL BYTE PTR[BX]



有符号数乘法指令

- 格式:

- IMUL OPRD

- 指令格式及对操作数的要求与MUL指令相同。

- 指令执行原理:

- ① 将两个操作数取补码（对负数按位取反加1，正数不变）；
- ② 做乘法运算；
- ③ 将乘积按位取反加1。

2. 除法指令

无符号除法指令

- 格式:
 - DIV OPRD

有符号除法指令

- 格式:
 - IDIV OPRD

除法指令的操作

若OPRD是字节数

- 执行: **AX/OPRD**
- 结果:
 - AL=商 AH=余数

若OPRD是双字节数

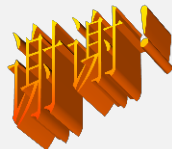
- 执行: **DXAX/OPRD**
- 结果:
 - AX=商 DX=余数

指令要求被
除数是除数
的双倍字长

算术运算指令小结

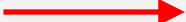
- 指令执行影响状态标志位
- 乘法指令执行结果为相乘数的双倍字长；除法指令要求被除数是除数的双倍字长。
- 例：
 - **MOV SI, 1200H**
 - **MOV WORD PTR[SI], 8765H**
 - **MOV AL, [SI]**
 - **INC SI**
 - **MUL BYTE PTR[SI]**

$AL \times [SI] \rightarrow AX = 3543H$



逻辑运算指令

逻辑运算

- 基本逻辑运算：
 - 与、或、非
 - 异或
- 逻辑运算指令  实现逻辑操作的指令
 - 与、或、非、异或

逻辑运算指令

■ 对操作数的要求：

- 大多与MOV指令相同。
- “非”运算指令要求操作数不能是立即数；

■ 对标志位的影响

- 除“非”运算指令，其余指令的执行都会影响除AF外的5个状态标志；
- 无论执行结果任何，都会使标志位OF=CF=0。
- “非”运算指令的执行不影响标志位。

1. “与”指令：

- 格式：

- AND OPRD1, OPRD2

- 操作：

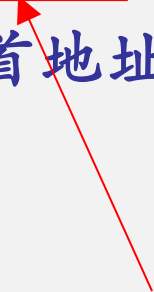
- 两操作数相“与”，结果送目标地址。

“与”指令的应用

- 实现两操作数按位相与的运算
 - `AND BL, [SI]`
- 使目标操作数的某些位不变，某些位清零
 - `AND AL, 0FH`
- 在操作数不变的情况下使CF和OF清零
 - `AND AX, AX`

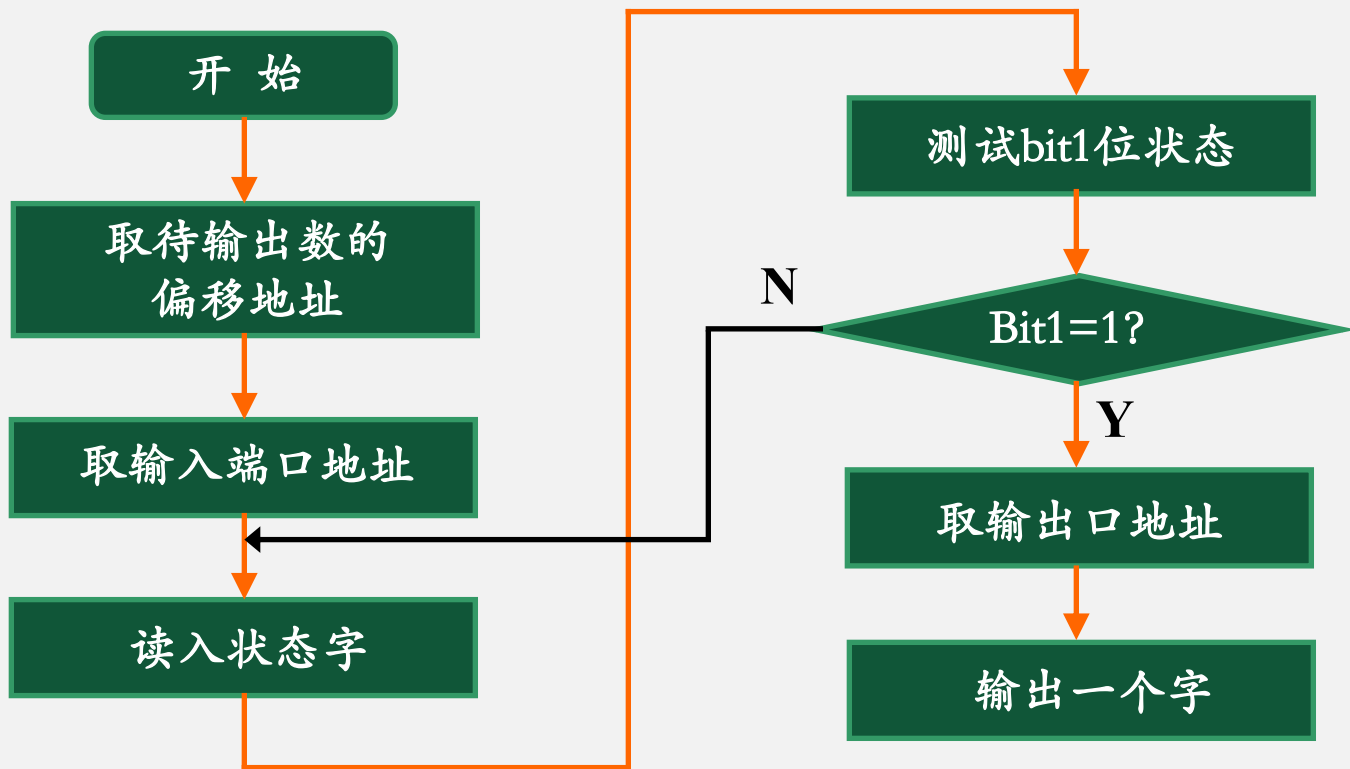
“与”指令应用例

- 从地址为3F8H端口中读入一个字节数，如果该数bit1位为1，则可从38FH端口将DATA为首地址的1个字输出，否则就不能进行数据传送。
- 要求：
 - 编写相应的程序段。



状态字

“与”指令应用例

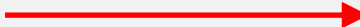


“与”指令应用例

MOV DX, 3F8H

WATT: IN AL, DX

AND AL, 02H

JZ WATT  **ZF=1转移**

MOV DX, 38FH

MOV AX, DATA

OUT DX, AX

2. “或” 运算指令

- 格式:

- OR OPRD1, OPRD2

- 操作:

- 两操作数相“或”，结果送目标地址

“或”指令的应用

- 实现两操作数相“或”的运算
 - **OR AX, [DI]**
- 使某些位不变，某些位置“1”
 - **OR CL, 0FH**
- 在不改变操作数的 情况下使OF=CF=0
 - **OR AX, AX**

“或”指令的应用例

OR AL, AL

JPE GOON

OR AL, 80H

GOON:



PF=1 转移

“或”指令的应用

将一个二进制
数9变为字符
‘9’

```
MOV AL, 9  
OR AL, 30H
```

如何实现？

3. “非” 运算指令

- 格式：
 - NOT OPRD
- 操作：
 - 操作数按位取反再送回原地址
- 注：
 - 指令的执行对标志位无影响
- 例：
 - **NOT BYTE PTR[BX]**

4. “异或” 运算指令

- 格式:

- XOR OPRD1, OPRD2

- 操作:

- 两操作数相“异或”，结果送目标地址

- 例:

- XOR BL, 80H

- XOR AX, AX

5. “测试” 指令

- 格式:

- TEST OPRD1, OPRD2

- 操作:

- 执行“与”运算，但运算的结果不送回目标地址。

- 应用:

- 常用于测试某些位的状态

例：

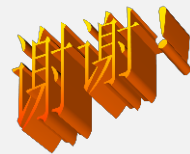
- 从地址为3F8H的端口中读入一个字节数，当该数的bit1，bit3，bit5位同时为1时，可从38FH端口将DATA为首地址的一个字输出，否则就不能进行数据传送。
- 编写相应的程序段。

源程序代码：

```
LEA SI, DATA
MOV DX, 3F8H
WATT: IN AL, DX

AND AL, 2AH
CMP AL, 2AH
JNZ WATT

MOV DX, 38FH
MOV AX, [SI]
OUT DX, AX
```



移位操作指令

移位操作指令

- 控制二进制位向左或向右移动的指令

 非循环移位指令
循环移位指令

移位操作指令说明

- 指令格式在形式上为双操作数，本质上为单操作数；
- 指令的目标操作数为被移动对象，源操作数为移动次数
 - 当目标为存储器操作数时，需要说明其字长
- 移动1位时由指令直接给出；移动两位及以上时，移位次数必须由CL指定。

**指令源操作数
只能是1或CL**

1. 非循环移位指令

- 逻辑左移
- 算术左移
- 逻辑右移
- 算术右移

算术左移和逻辑左移

算术左移指令：

SAL OPRD, 1
SAL OPRD, CL } 有符号数



逻辑左移指令：

SHL OPRD, 1
SHL OPRD, CL } 无符号数

逻辑右移

格式:

SHR OPRD, 1

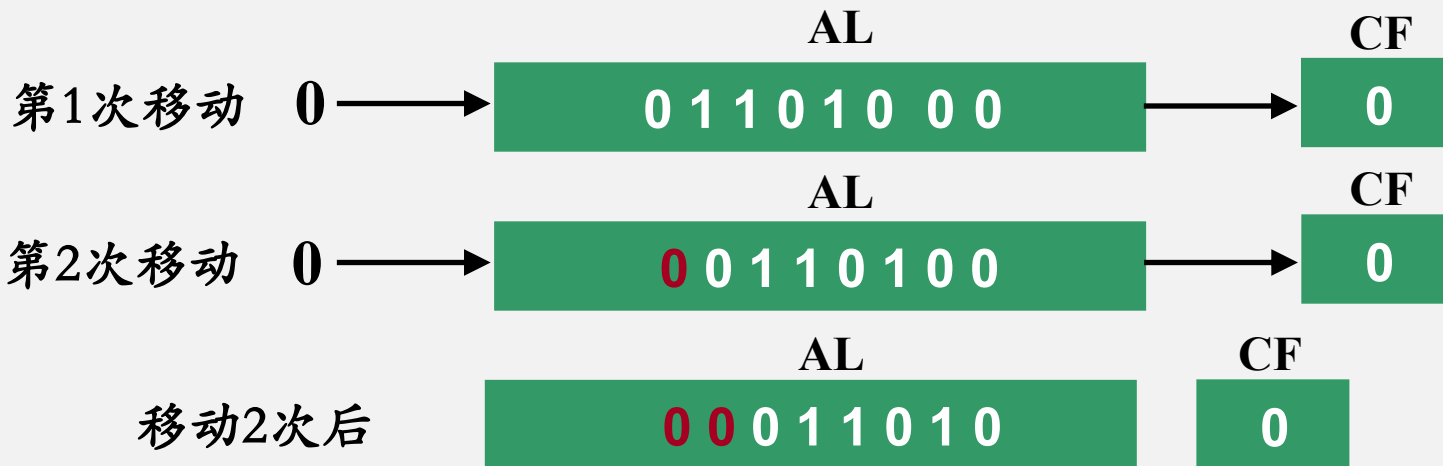
SHR OPRD, CL

无符号数
的右移



逻辑右移例：

- MOV AL, 68H
- MOV CL, 2
- SHR AL, CL



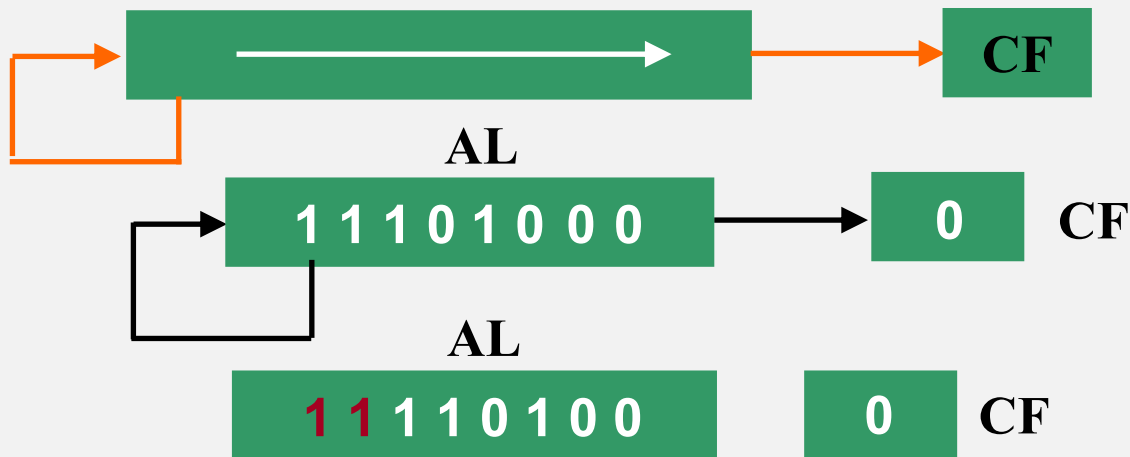
算术右移

格式:

SAR OPRD, 1

SAR OPRD, CL

有符号数
的右移



非循环移位指令的应用

- 左移可实现乘法运算
- 右移可实现除法运算

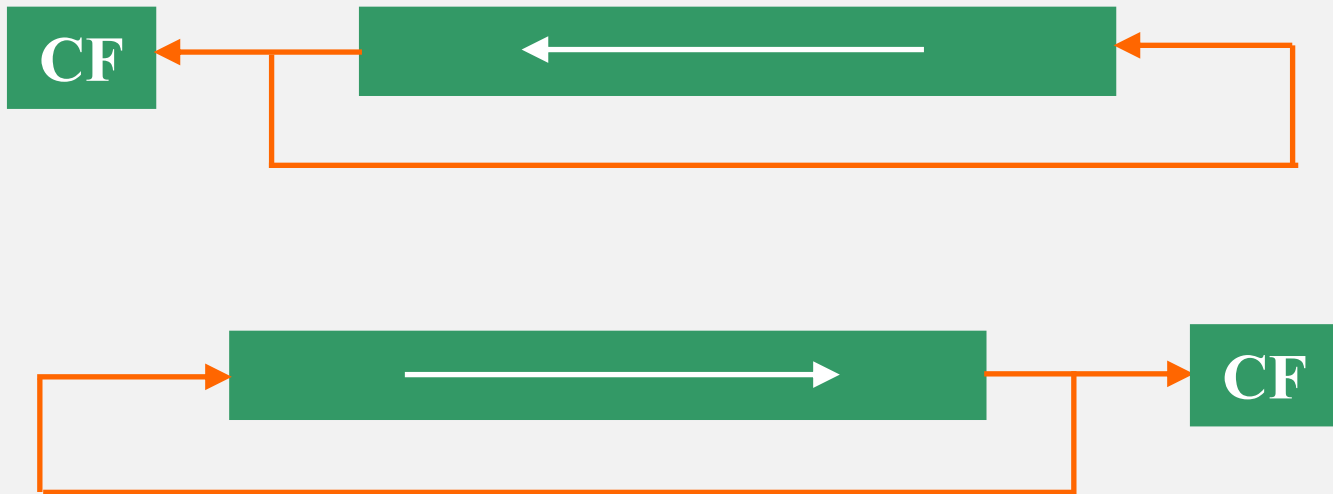
2. 循环移位指令

不带进位位的循环移位 { 左移 ROL
右移 ROR

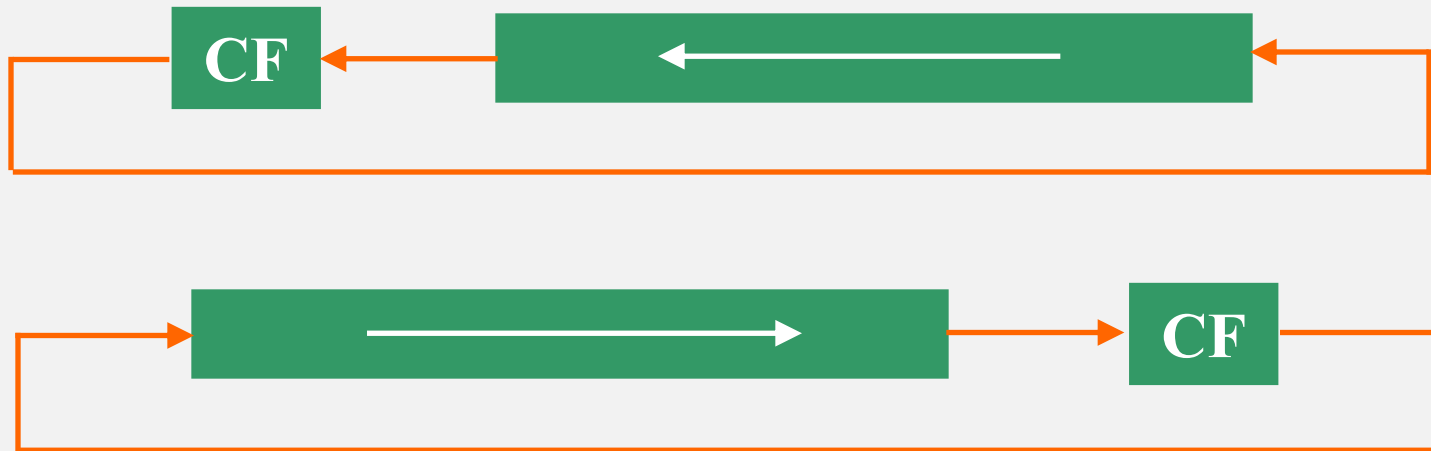
带进位位的循环移位 { 左移 RCL
右移 RCR

指令格式、对操作数的要求与非循环移位指令相同

不带进位的循环移位



带进位位的循环移位



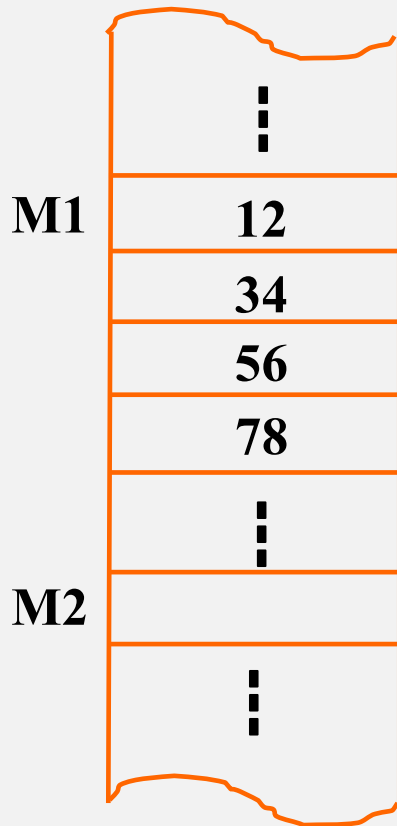
循环移位指令的应用

- 用于对某些位状态的测试;
- 高位部分和低位部分的交换;
- 与非循环移位指令一起组成32位或更长字长数的移位。

例：

在内存数据段M1为首地址的4个单元中存放了4个压缩BCD码。

要求：将这4个压缩BCD码分别转换为ASCII码，并将转换结果存放在同一逻辑段、M2为首的单元中。



例：

■ 题目分析：

- 压缩BCD码是用4位二进制码表示1位十进制数
- 转换ASCII码时需要分别转换高4位（十位数）和低4位（个位数）
- 0~9的ASCII码的高4位均为0011（03H）
- 转换低4位时应先使高4位清零，转换高4位时须先将高4位移动到低4位的位置。

1字节表示2

位压缩BCD



程序例

LEA SI,M1

LEA DI,M2

MOV CH,4

Next: MOV AL,[SI]

MOV BL,AL

AND AL,0FH

OR AL,30H

MOV [DI],AL

INC DI

MOV AL,BL

MOV CL,4

SHR AL,CL

OR AL,30H

MOV [DI],AL

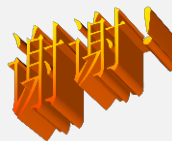
INC DI

INC SI

DEC CH

JNZ Next

HLT



串 操 作

一、串操作指令说明

串操作指令说明

- 针对数据块或字符串的操作
- 可实现存储器到存储器的数据传送；
- 待操作的数据串称为源串，目标地址称为目标串。

将M1的数据送到
M2起始的区域中

原串

M1

12H

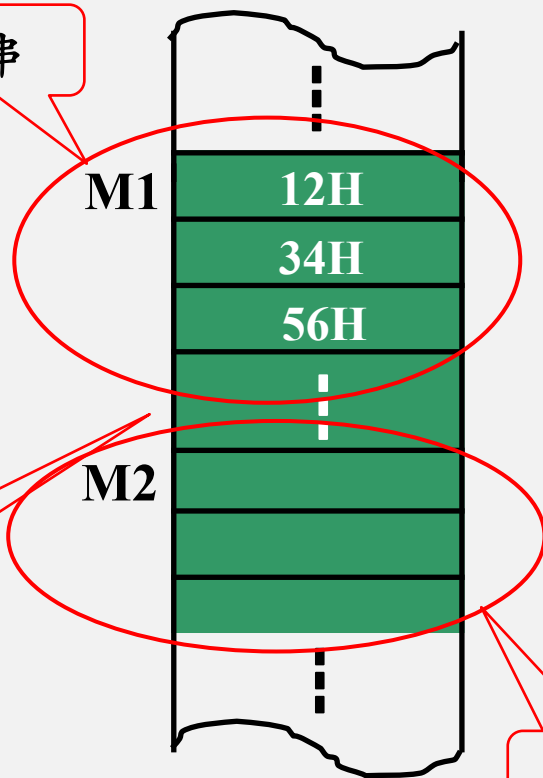
34H

56H

⋮

M2

目标串



串操作指令说明

- 串操作指令的操作对象是多个字节数（一串字符或数据），因此，指令的执行需要确定：
 - 串所在的区域
 - 串的首地址（原串、目标串起始地址）
 - 串长度（大小）
 - 串的操作方向

串操作指令的要求

通过增加**重复前缀**，
可以实现对CX值的
自动修改

■ 串所在区域及首地址：

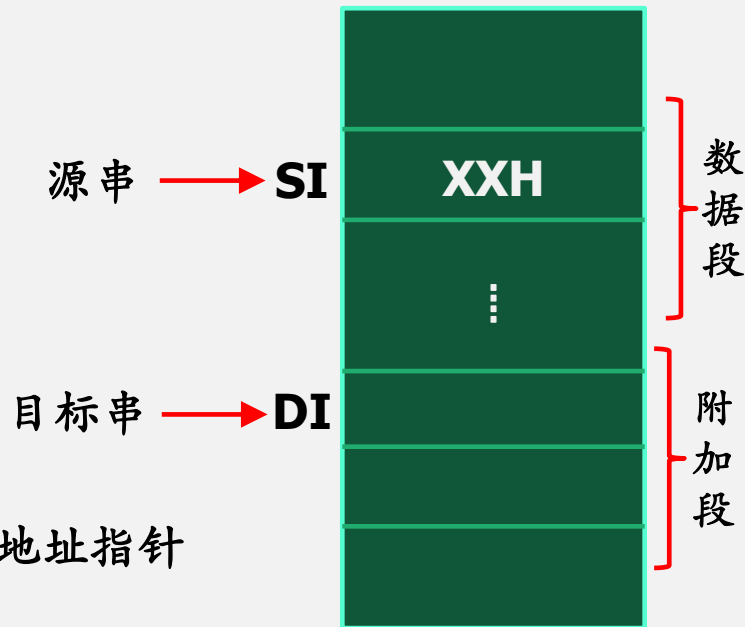
- 源串**一般**存放在数据段，偏移地址由SI指定。
- 目标串**必须**在附加段，偏移地址由DI指定。

■ 串长度：

- 串长度值由CX指定

■ 串的操作方向：

- 由DF标志位决定。指令根据DF状态自动修改地址指针
 - **DF=0** ———→ 增地址方向
 - **DF=1** ———→ 减地址方向



重复前缀

■ 无条件重复

■ REP

- 当 $CX \neq 0$ 时，REP 后的指令将继续重复执行
- 常用于传送类指令前  未传完则继续传送

■ 条件重复

- 条件前缀常用于运算类指令前，当：

1) 操作未结束 AND 结果=0 或者

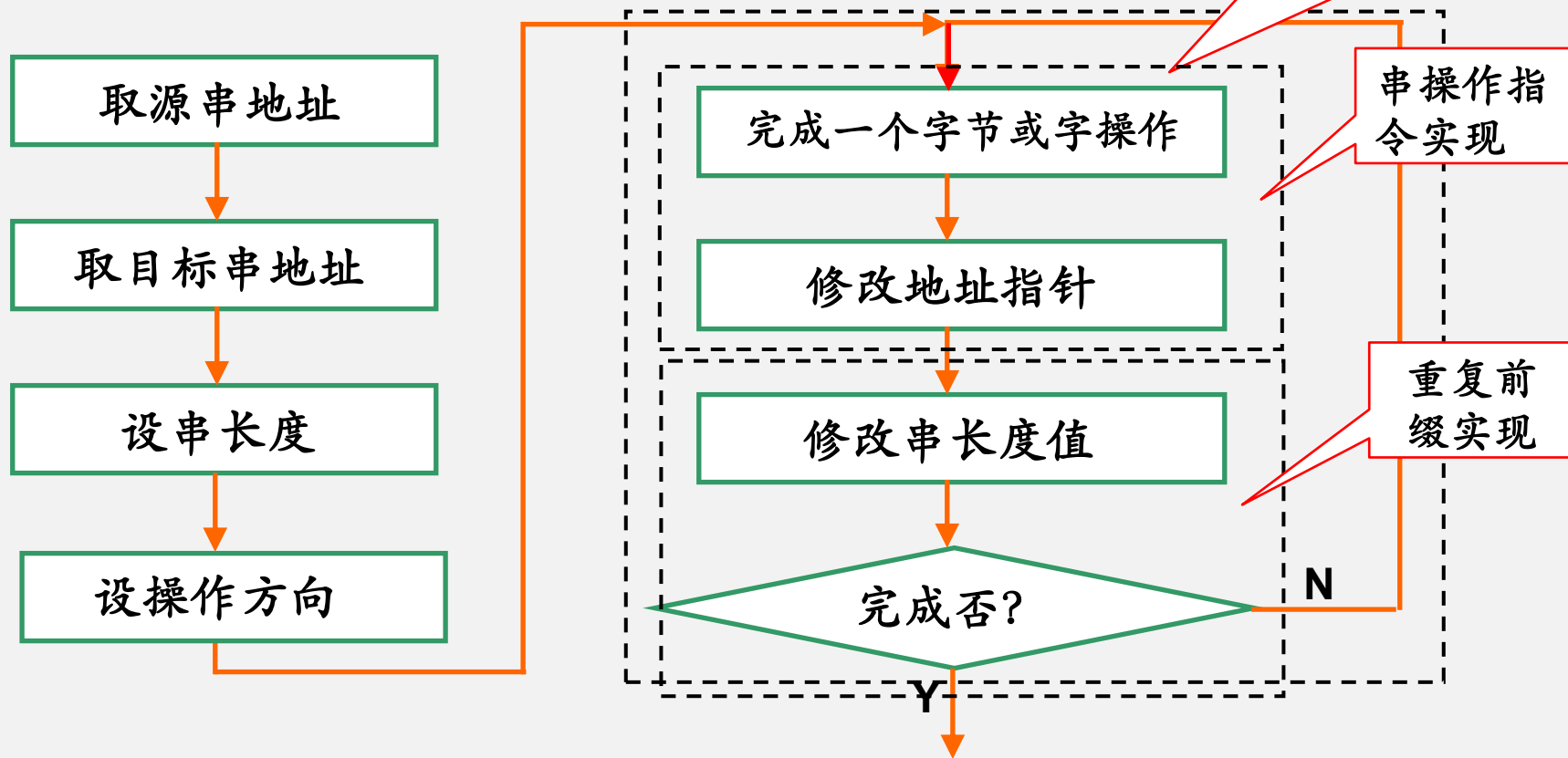
- 2) 操作未结束 AND 结果 $\neq 0$

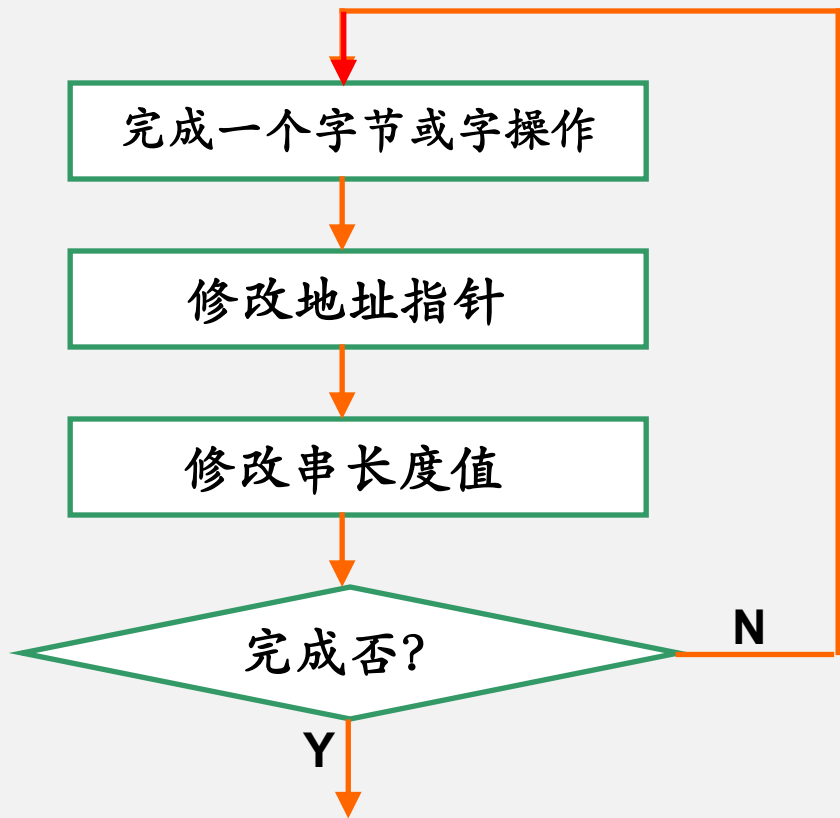
使其后的指令继续重复执行。

串操作指令

- 串传送 MOVS
- 串比较 CMPS
- 串扫描 SCAS
- 串装入 LODS
- 串送存 STOS

串操作指令流程





CX=0

SI

XXH

SI

XXH

SI

DI

XXH

DI

DI

◆ 若按增地址方向操作，串操作结束时：

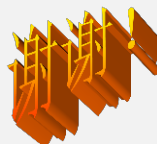
◆ 串传送指令：指针将指向串尾+1

◆ 串比较类指令：指针将指向结束位+1

◆ 若按减地址方向操作，串操作结束时：

◆ 串传送指令：指针将指向串尾-1

◆ 串比较类指令：指针将指向结束位-1



二、串操作指令

1. 串传送指令

■ 功能：

- 将源数据串传送到目标地址

此格式仅用于源操作数需段重设的情况下

■ 格式：

① **MOVS OPRD1, OPRD2**

MOVS [DI],[SI]

② **MOVSB**

按字节传送

③ **MOVSW**

按字传送

■ 串传送指令常与无条件重复前缀连用

串传送指令例

- 分别用MOV指令和MOVS指令编写将200个字节数据从内存数据段MEM1为首地址的区域送到同一逻辑段MEM2为首地址的区域中的程序。

```
LEA SI, MEM1
LEA DI, MEM2
MOV CX, 200
NEXT: MOV AL, [SI]
      MOV [DI], AL
      INC SI
      INC DI
      DEC CX
      JNZ NEXT
      HLT
```

```
LEA SI, MEM1
LEA DI, MEM2
MOV CX, 200
CLD
REP MOVSB
HLT
```

2. 串比较指令

- 功能：
 - 用于实现两个数据串的比较
- 操作：
 - 目标串-源串，结果不写回目标地址
 - 常与条件重复前缀连用
- 格式：
 - ① CMPS OPRD1, OPRD2
 - ② CMPSB
 - ③ CMPSW
- 前缀的操作对标志位不影响

串比较指令例

- 测试上例中200个字节数据是否传送正确。

结束串比较指令的条件:

- ① $CX=0$;
- ② $CX \neq 0$, 但 $ZF=0$

```
1.      LEA SI, MEM1
2.      LEA DI, MEM2
3.      MOV CX, 200
4.      CLD
5.      REPE CMPSB
6.      JZ STOP  → 两数据串相同
7.      DEC SI   → 指向存放不相
8.      MOV AL, [SI]
9.      MOV BX, SI
10.     STOP: HLT
```

获取不相等数据及
存放该数据的地址

串扫描指令

3. 串扫描指令

常用于在指定存储区域中寻找某个关键字

- 格式:

- SCAS OPRD

目 标
操作数

- SCASB

- SCASW

- 执行与CMPS指令相似的操作，区别是：

- 这里的源操作数是AX或AL

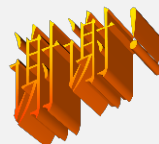
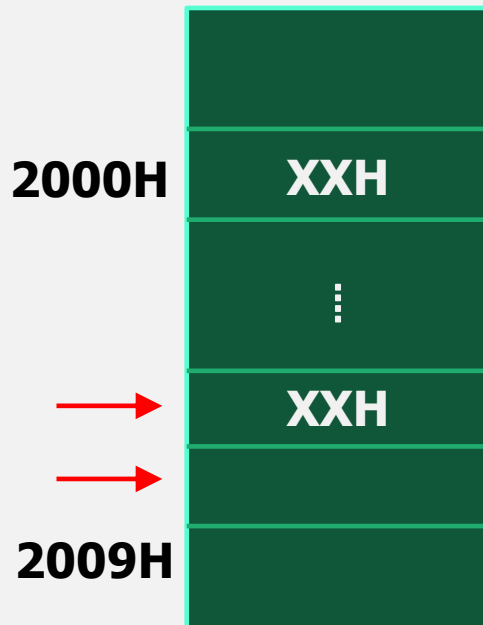
串扫描指令应用例：

- 在ES段中从2000H单元开始存放了10个字符，寻找其中有无字符“A”。若有则记下搜索次数，将搜索次数写入到DATA1单元，并将存放“A”的地址写入DATA2单元。

串扫描指令应用例：

```
MOV DI, 2000H
MOV BX, DI
MOV CX, 0AH
MOV AL, 'A'
CLD
REPNZ SCASB
```

```
                JZ FOUND
                MOV DI, 0
                JMP DONE
FOUND: DEC DI
        MOV DATA2, DI
        INC DI
        SUB DI, BX
DONE:  MOV DATA1, DI
        HLT
```



串装入与串存储指令

4. 串装入指令

- 格式:

- **LODS OPRD**
- **LODSB**
- **LODSW**

源操作数

- 操作:

- 对字节: **AL** ← **[DS:SI]**
- 对字: **AX** ← **[DS:SI]**

串装入指令

- 用于将内存某个区域的数据串依次装入累加器，以便显示或输出到接口。
- **LODS**指令一般不加重复前缀。

5. 串存储指令

■ 格式:

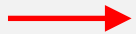
- **STOS OPRD**

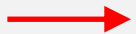
- **STOSB**

- **STOSW**

目 标
操作数

■ 操作:

- 对字节: AL  [ES:DI]

- 对 字: AX  [ES:DI]

串存储指令的应用

- 常用于将内存某个区域置同样的值
- 此时：
 - 将待送存的数据放入AL（字节数）或AX（字数据）；
 - 确定操作方向（增地址/减地址）和区域大小（串长度值）；
 - 使用串存储指令+无条件重复前缀，实现数据传送。

串存储指令例

- 将6000H:1200H单元开始的100个字存储单元内容清零。
- 分析：
 - 可用MOV指令或串存储指令实现

MOV AX, 6000H

MOV ES, AX

MOV DI, 1200H

MOV CX, 100

CLD

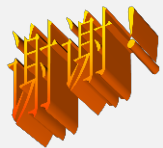
MOV AX, 0

REP STOSW

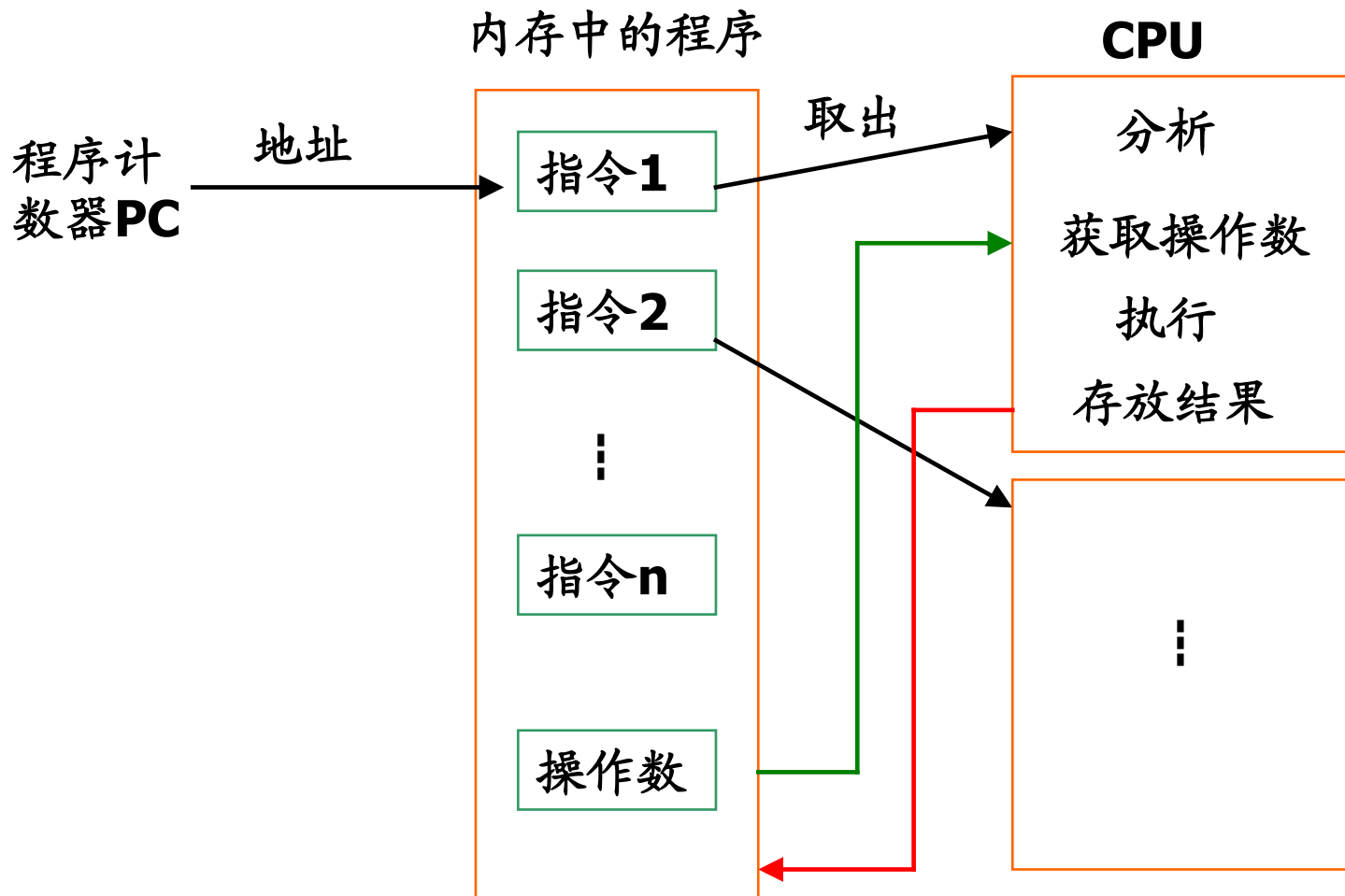
HLT

串操作指令应用注意事项

- 需要定义附加段
 - 目标操作数必须在附加段
- 需要设置数据的操作方向
 - 确定DF的状态
- 源串和目标串指针分别为SI和DI
- 串长度值必须由CX给出
- 注意重复前缀的使用方法
 - 传送类指令前加无条件重复前缀
 - 串比较类指令前加条件重复前缀，但前缀不影响ZF状态



程序控制指令



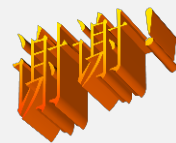
程序控制类指令

- 程序控制类指令以“隐含”的方式修改CS和IP，以实现控制程序走向的目的（Intel指令集不允许由指令直接修改CS和IP）
- 通过修改IP或CS和IP，实现程序的三种基本控制结构
 - 顺序，选择（分支），循环
- 学习程序控制类指令需要重点关注：

如何实现CS和IP的修改？

程序控制类指令

- 转移指令
- 循环控制
- 过程调用
- 中断控制



一、转移指令

转移指令

通过修改指令的偏移地址或段地址及偏移地址实现程序的转移

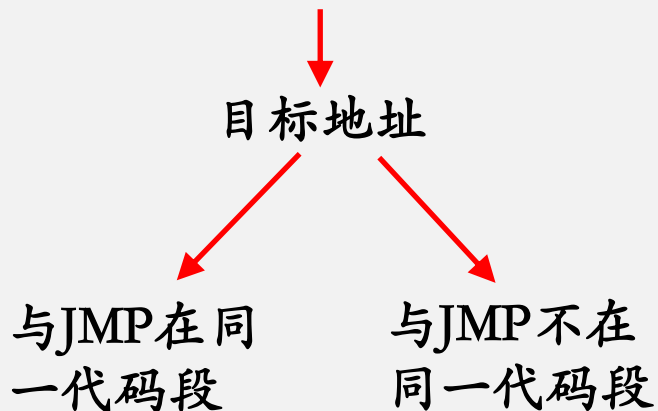
- 无条件转移指令 → 无条件转移到目标地址
- 条件转移指令 → 当具备一定条件时转移到目标地址

通常指状态标志位

1. 无条件转移指令

■ 格式:

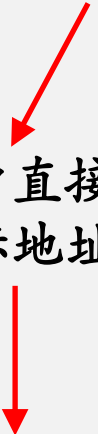
■ **JMP OPRD**



可以实现在当前
代码段内或段间
转移

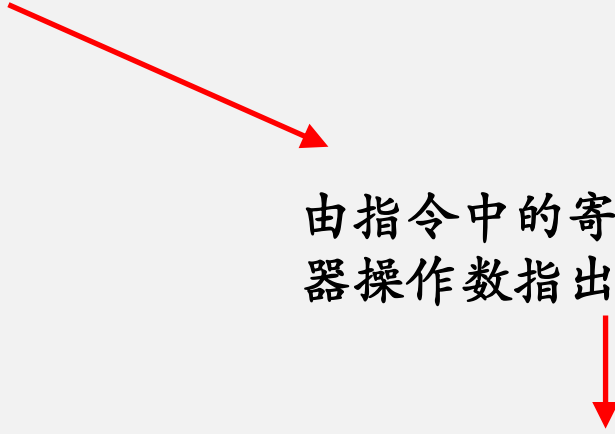
1) 无条件段内转移

- 转移的目标地址在当前代码段内，段地址不改变。
- 即：目标地址是16位偏移地址。



指令中直接给出目标地址

段内直接转移

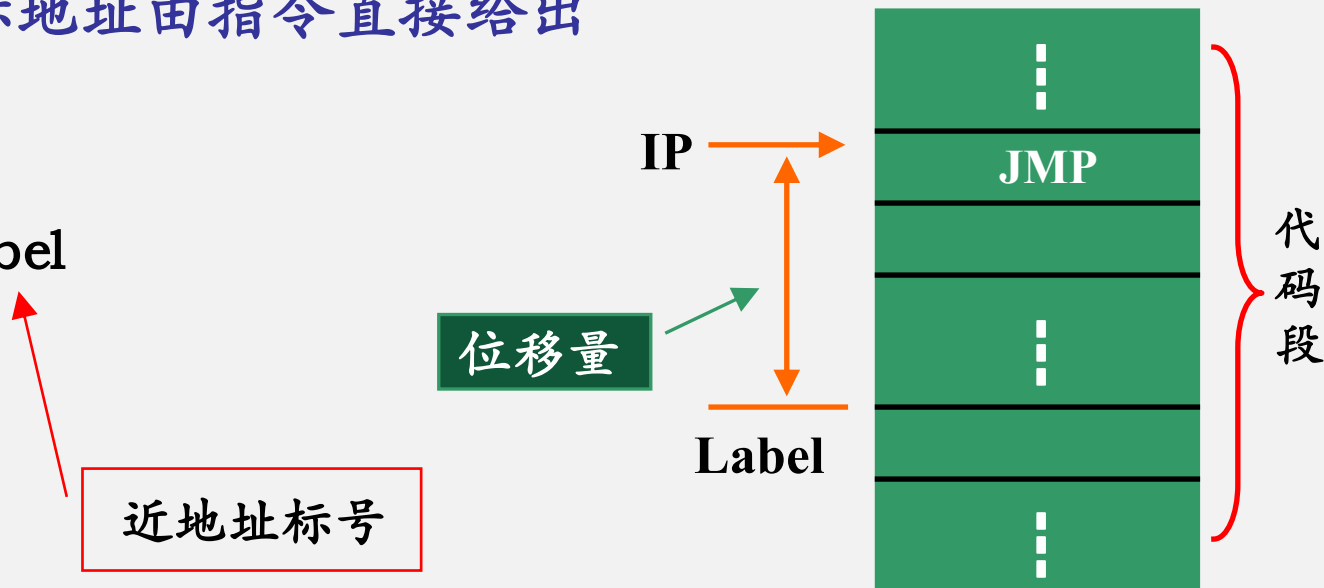


由指令中的寄存器或存储器操作数指出目标地址

段内间接转移

段内直接转移

- 转移的目标地址由指令直接给出
- 格式:
 - `JMP Label`



下一条要执行指令的偏移地址 = 当前IP + 位移量

段内间接转移

- 段内间接转移

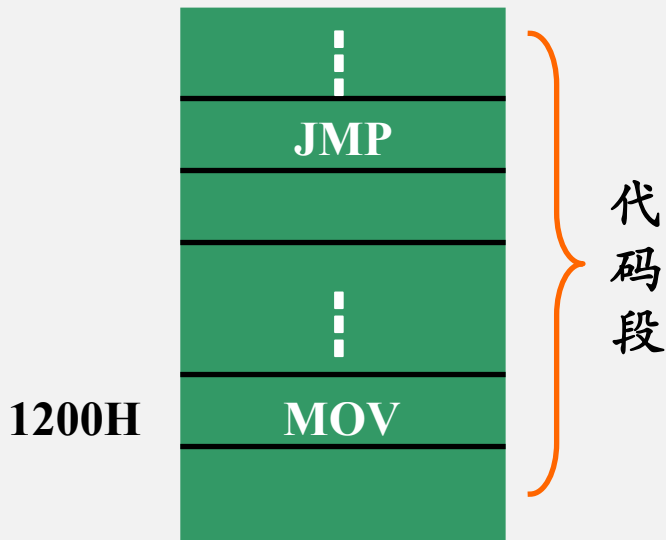
- 转移的目标地址存放在某个16位寄存器或存储器的某两个单元中

- 例：

- `MOV BX, 1200H`
- `JMP BX`

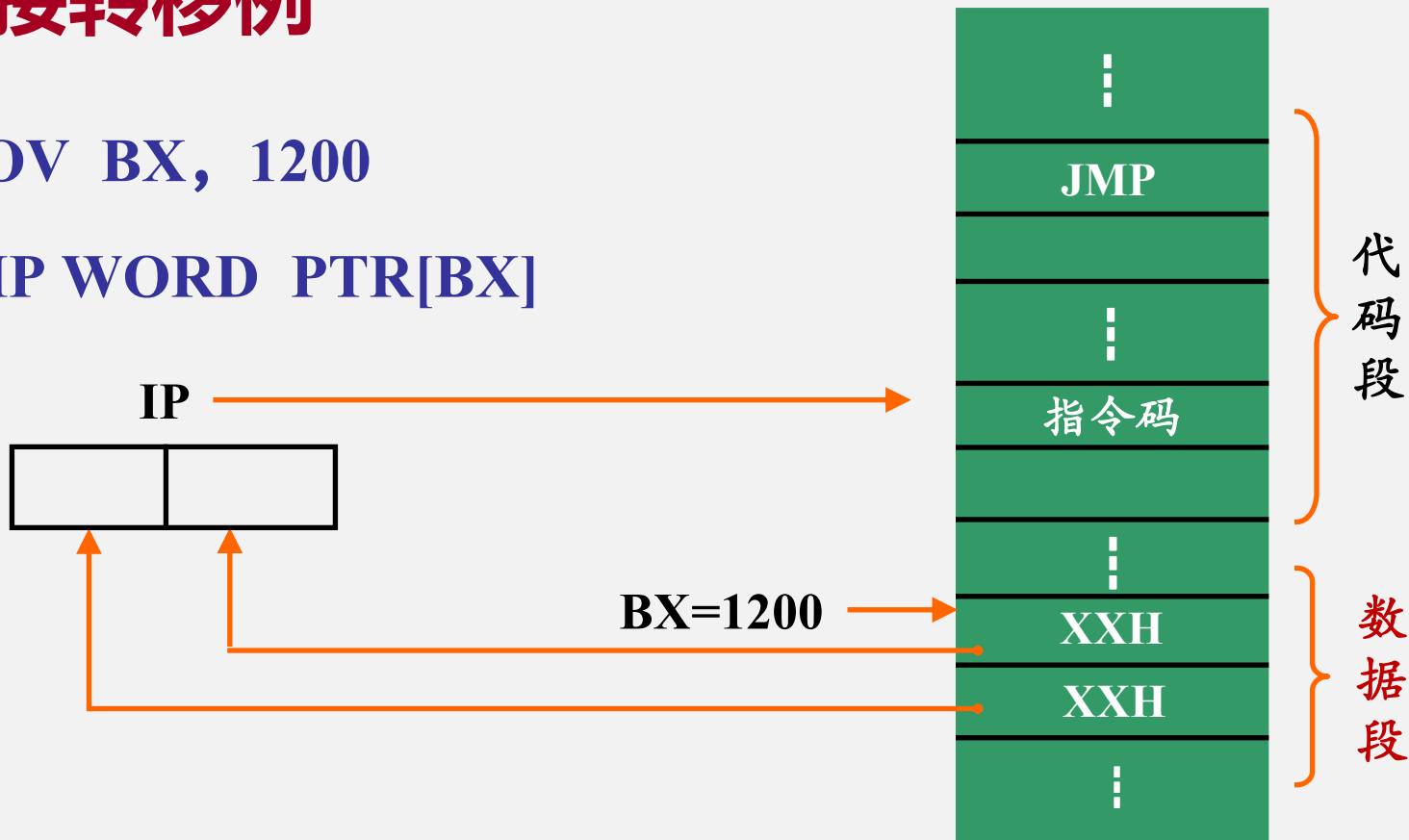
- 执行完上述指令后：

- `IP=1200H`



段内间接转移例

- `MOV BX, 1200`
- `JMP WORD PTR[BX]`



2) 无条件段间转移

- 转移的目标地址不在当前代码段内。
- 目标地址为32位，包括段地址和偏移地址。

指令中直接给出目标地址

段间直接转移

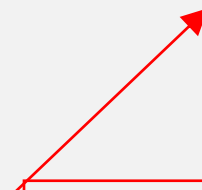
由指令中的32位存储器操作数指出目标地址

段间间接转移

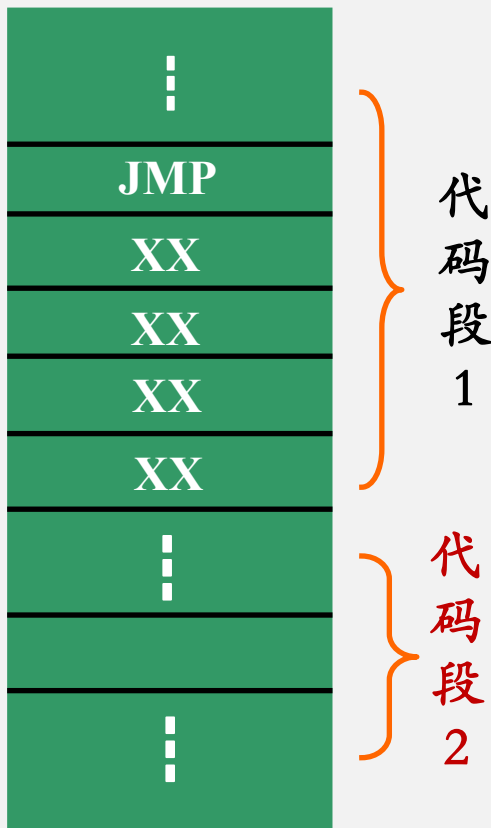
段间直接转移

- 段间直接转移
 - 转移的目标地址由指令直接给出
- 格式:
 - **JMP FAR Label**

远地址标号



Label



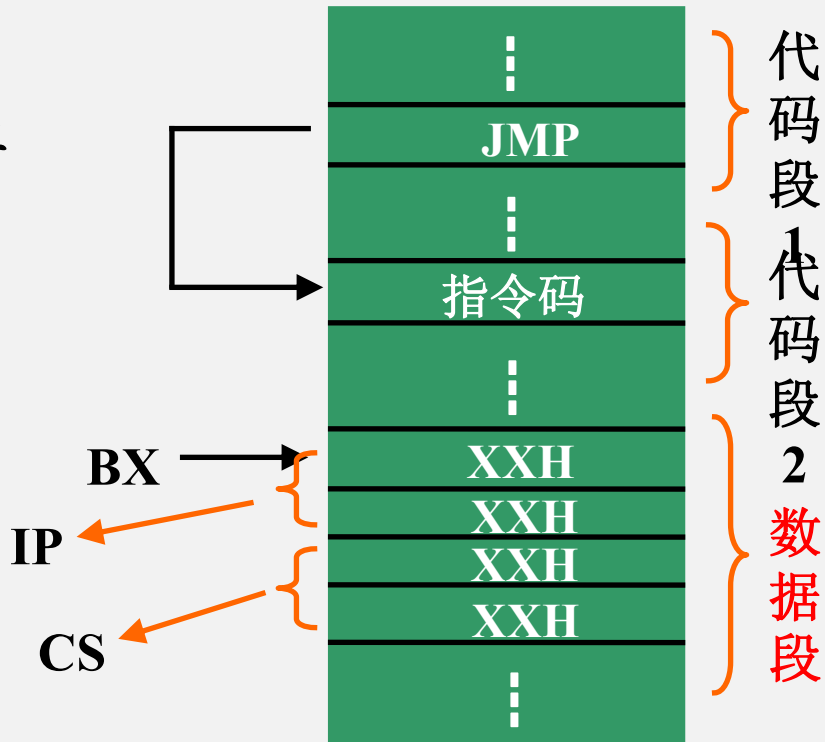
段间间接转移

■ 段间间接寻址

- 转移的目标地址由指令中的32位操作数给出
- 32位目标地址须存放于内存中

■ 例：

- **JMP DWORD PTR[BX]**



无条件转移指令例

CS: IP

(1) 2000: 0100 MOV AX,1200H

(2) 2000: 0103 JMP NEXT

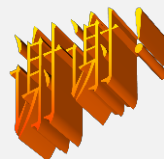
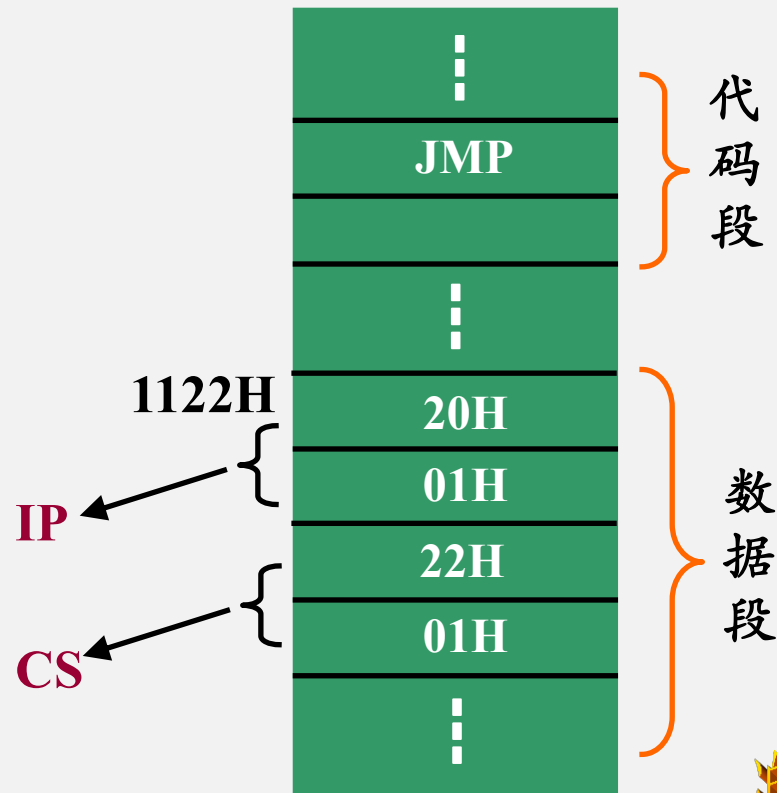
(3) 2000: 0120 NEXT: MOV BX,1200H

(4) JMP BX

(5) 2000: 1200

无条件转移指令例

- `MOV SI, 1122H`
 - `MOV WORD PTR[SI], 0120H`
 - `ADD SI, 2`
 - `MOV WORD PTR[SI], 0122H`
- `JMP DWORD PTR[SI-2]`**



程序代码

```
START: XOR AL, AL
        MOV PLUS, AL
        MOV MINUS, AL
        MOV ZERO, AL
        LEA SI, TABLE
        MOV CL, 100
        CLD
```

```
CHECK: LODSB
```

```
        OR AL, AL
```

```
        JS X1
```

```
        JZ X2
```

```
        INC PLUS
```

```
        JMP NEXT
```

```
X1: INC MINUS
```

```
        JMP NEXT
```

```
X2: INC ZERO
```

```
NEXT: DEC CL
```

```
        JNZ CHECK
```

```
        HLT
```


2. 条件转移指令

- 在满足一定条件下，程序转移到目标地址继续执行
- 条件转移指令均为段内短转移，即转移范围为：

-128-----+127

条件转移指令

指令名称	汇编格式	转移条件	备注
CX内容为0转移	JCXZ target	CX=0	
大于/不小于等于转移	JG/JNLE target	SF=OF且ZF=0	带符号数
大于等于/不小于转移	JGE/JNL target	SF=OF	带符号数
小于/不大于等于转移	JL/JNGE target	SF≠OF且ZF=0	带符号数
小于等于/不大于转移	JLE/JNG target	SF≠OF 或ZF=1	带符号数
溢出转移	JO target	OF=1	
不溢出转移	JNO target	OF=0	
结果为负转移	JS target	SF=1	
结果为正转移	JNS target	SF=0	
高于/不低于等于转移	JA/JNBE target	CF=0且ZF=0	无符号数
高于等于/不低于转移	JAE/JNB target	CF=0	无符号数
低于/不高于等于转移	JB/JNAE target	CF=1	无符号数
低于等于/不高于转移	JBE/JNA target	CF=1或ZF=1	无符号数
进位转移	JC target	CF=1	
无进位转移	JNC target	CF=0	
等于或为零转移	JE/JZ target	ZF=1	
不等于或非零转移	JNE/JNZ target	ZF=0	
奇偶校验为偶转移	JP/JPE target	PF=1	
奇偶校验为奇转移	JNP/JPO target	PF=0	

条件转移指令

■ 基于1个标志位状态实现转移的指令

■ JC/JNC

- 判断CF的状态。常用于两个无符号数大小比较

■ JZ/JNZ

- 判断ZF的状态。常用于循环体的结束判断

■ JO/JNO

- 判断OF的状态。常用于有符号数溢出的判断

■ JP/JPE

- 判断PF的状态。用于判断运算结果低8位中1的个数是否为偶数

■ JS /JNS

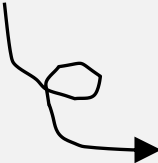
- 判断SF的状态。常用于判断数的性质

条件转移指令

- 基于2个或3个标志位状态实现转移的指令：
 - JA/JAE/JB/JBE
 - 判断CF或CF+ZF的状态。常用于无符号数大小的比较
 - JG/JGE/JL/JLE
 - 判断SF+OF或SF+OF+ZF的状态。常用于有符号数大小的比较
- 基于CX内容转移的指令
 - JCXZ
 - 可根据指令执行后CX的结果实现转移

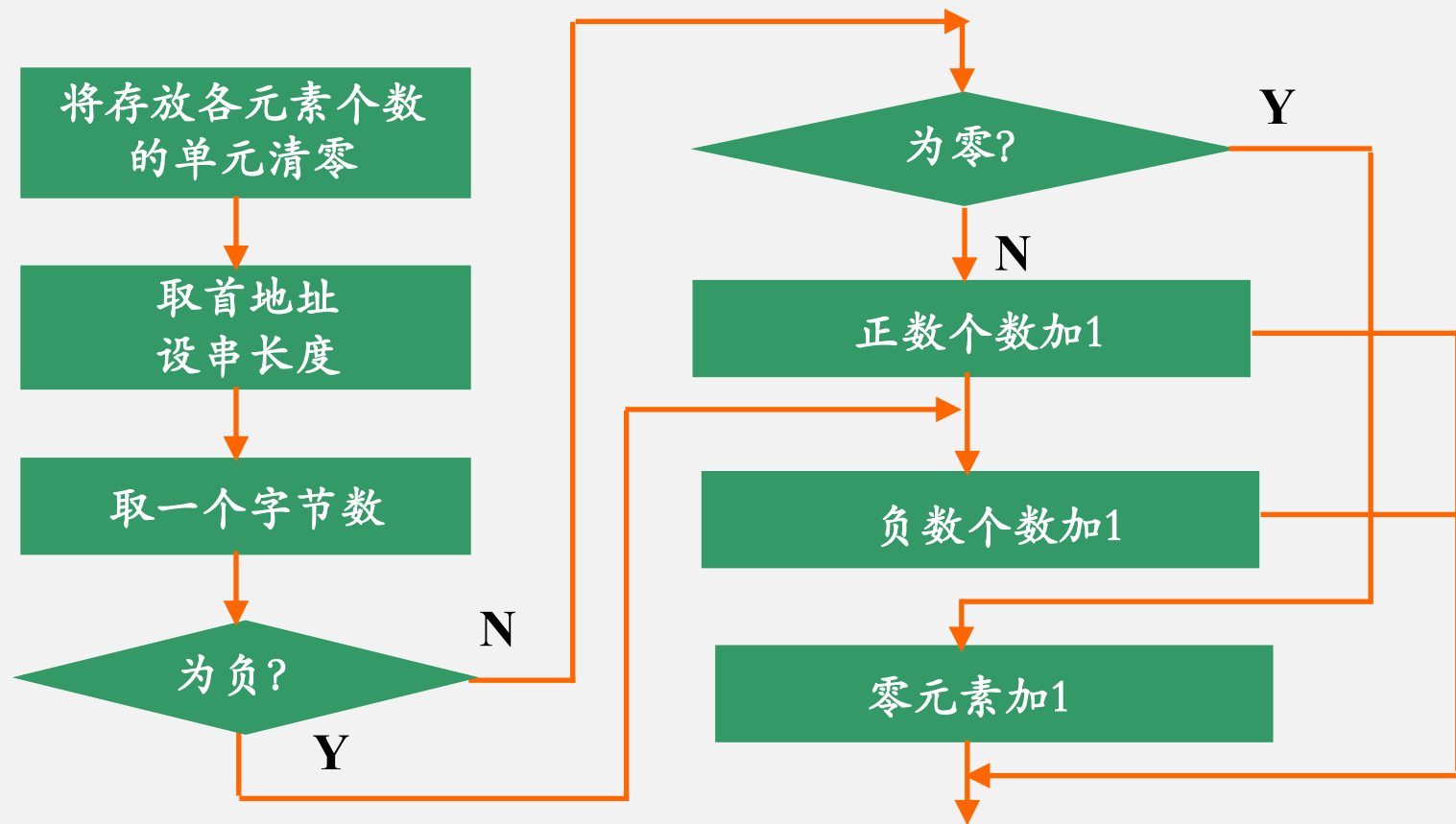
转移指令例

- 统计内存数据段中以TABLE为首地址的100个8位带符号数中正数、负数和零元数的个数。
- 基本思路：
 - 可先将存放统计值的单元（或寄存器）清零
 - 读取一个数，通过标志位的状态判断数的性质
 - 最高位为1，则为负数
 - 最高位为0，则为正数或零



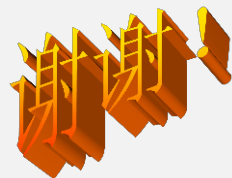
如何既不改变数本身，又能影响标志位从而获得数的性质？

转移指令例（流程图）



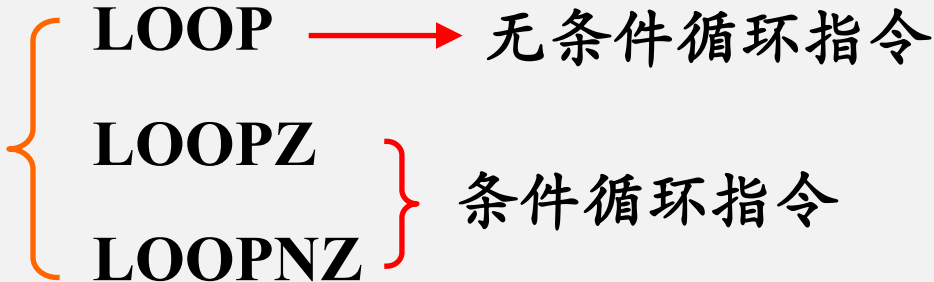
程序代码

```
START: XOR AL, AL
        MOV PLUS, AL
        MOV MINUS, AL
        MOV ZERO, AL
        LEA SI, TABLE
        MOV CL, 100
        CLD
CHECK:  LODSB
        OR AL, AL
        JS X1
        JZ X2
        INC PLUS
        JMP NEXT
X1:     INC MINUS
        JMP NEXT
X2:     INC ZERO
        NEXT: DEC CL
        JNZ CHECK
        HLT
```



二、 循环控制指令

循环控制指令

- 循环范围：
 - 以当前IP为中心的-128~+127范围内循环。
- 循环次数由CX寄存器指定。
- 循环指令：
 - A diagram showing the classification of loop instructions. On the left, a large orange curly bracket groups three instructions: LOOP, LOOPZ, and LOOPNZ. To the right of LOOP, a red arrow points to the text '无条件循环指令' (Unconditional loop instruction). To the right of the LOOPZ and LOOPNZ instructions, a red curly bracket groups them, with the text '条件循环指令' (Conditional loop instruction) to its right.

无条件循环指令

- 格式:

- `LOOP LABEL`

- 循环条件:

- `CX  $\neq$  0`

- 操作:

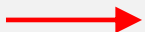
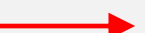
完全相当于 { `DEC CX`
`JNZ 符号地址`

条件循环指令

- 功能:

- 先使CX-1，再根据CX中的值及ZF值来决定是否继续循环

- 格式:

- **LOOPZ Label**  继续循环的条件: **CX≠0**, 且**ZF=1**
- **LOOPNZ Label**  继续循环的条件: **CX≠0**, 且**ZF=0**

例：

- 在以DATA为首地址的内存数据段中，存放有200个16位有符号数，试找出其中最大和最小的符号数，并分别放在MAX和MIN为首的内存单元中。
- 题目分析：
 - 先取数据块中第1个数，将其同时暂存于MAX和MIN中；
 - 循环读取其它数并分别与MAX和MIN中的数进行比较
 - 若大于则取代原MAX中的数；若小于MIN内容，则将新数放于MIN中。直到结束。

注：有符号数比较应采用JG和JL等用于符号数的条件转移指令

```
START: LEA SI, DATA
      MOV CX, 200
      CLD
      LODSW      → 读取2B到AX
      MOV MAX, AX
      MOV MIN, AX
      DEC CX
NEXT:  LODSW
      CMP AX, MAX
      JG LARGE
      CMP AX, MIN
      JL SMALL
      JMP GOON
LARGE: MOV MAX, AX
      JMP GOON ;
SMALL: MOV MIN, AX
GOON:  LOOP NEXT
      HLT
```



三、过程调用指令

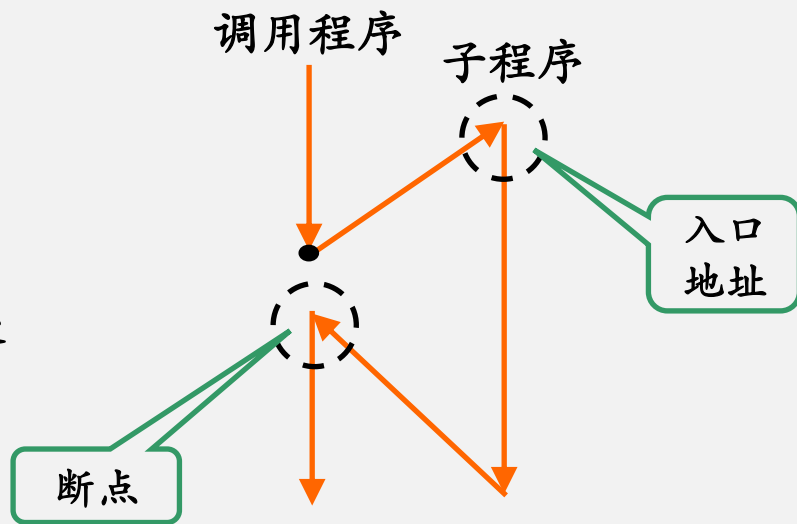
过程调用

■ 过程调用指令

- 用于调用一个子过程

■ 与转移指令的比较

- 子过程执行结束后要返回原调用处
 - 必须保护返回地址



调用指令的执行过程

① 保护断点

- 将调用指令的下一条指令的地址（断点）压入堆栈

② 获取子过程的入口地址

- 子过程第1条指令的偏移地址

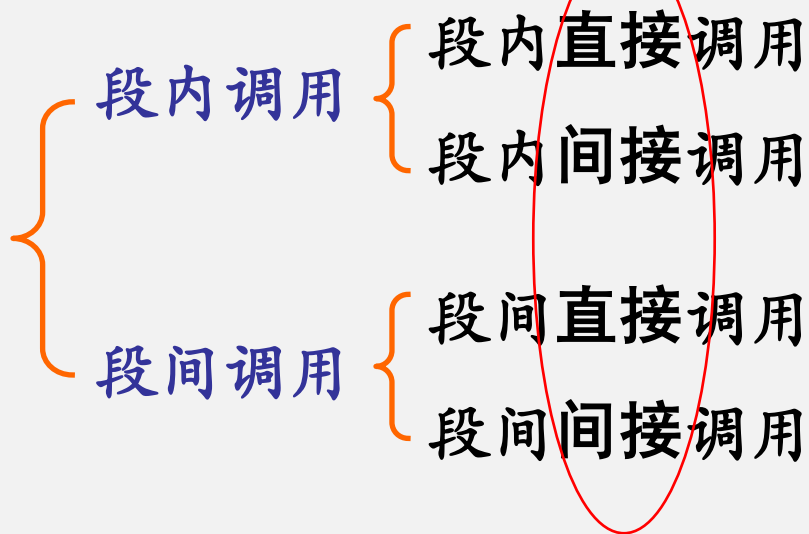
③ 执行子过程

- 功能实现，参数的保存及恢复

④ 恢复断点，返回原程序。

- 将断点偏移地址由堆栈弹出

过程调用



直接调用：

指令中直接给出子过程的入口地址

间接调用：

由内存获得子过程的入口地址

1. 段内调用

- 被调用程序与调用程序在同一代码段

- 调用前只需保护断点的偏移地址

- 格式:

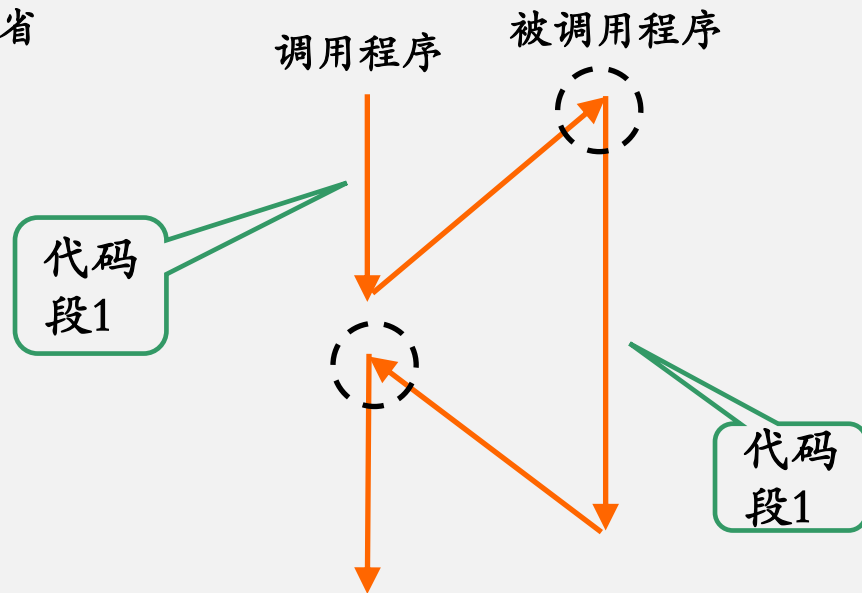
- CALL NEAR PROCC

近过程名

可以缺省

- 执行过程:

- 将断点的偏移地址压入堆栈
- 根据过程名找子程序入口



段内调用例

(1) CALL TIMER → 直接调用

调用名字为TIMER
的程序的入口
地址在内存中

(2) CALL WORD PTR[SI] → 间接调用

设: SI=1200H
CS=6000H

执行第 (2) 条指令后:

CS = 6000H

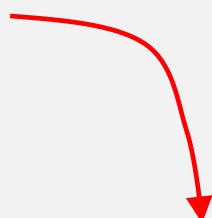
IP = 3344H



2. 段间调用

- 子过程与原调用程序不在同一代码段

- 先将断点的CS压栈，再压入IP。



调用前需保护断点的
段基址和偏移地址

段间调用例

■ 格式例:

■ CALL FAR TIMRE

■ CALL DWORD PTR[SI]

调用名为TIMER
的远过程

32位存储器操
作数，表示远
过程调用

CS

IP

CALL

⋮

XXH

XXH

XXH

XXH

代码段

数据段

3. 返回指令

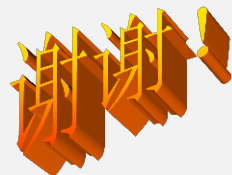
- 功能:

- 从堆栈中弹出断点地址，返回原程序

- 格式:

- RET

子程序的最后一条指令必须是RET



第3章作业

- 作业请从电子教室网站下载
- 本章书后全部题目均可作为思考题

四、中断控制指令

四、中断指令

■ 中断的概念

- 某种异常或随机事件使处理器暂时停止正在运行的程序，转去执行一段特殊处理程序，并在处理结束后返回原程序被中断处继续执行的过程。

中断源

■ 中断指令：

- 引起CPU产生一次中断的指令

中断与过程调用：

■ 相似点：

- 从一个正在执行的过程转向另一个过程（处理程序），并在执行完后返回原程序继续执行

■ 区别：

- 中断是随机事件或异常事件引起，调用是事先已在程序中安排好；
- 调用指令在指令中直接给出子程序入口地址，中断指令只给出中断向量码，入口地址则在向量码指向的内存单元中。
- 调用可以是近过程调用或远过程调用，中断处理程序均为远过程；
- 响应中断请求不仅要保护断点地址，还要保护 FLAGS 内容。

1. 中断指令

- 格式:

INT n

中断类型码
n=0 ~ 255

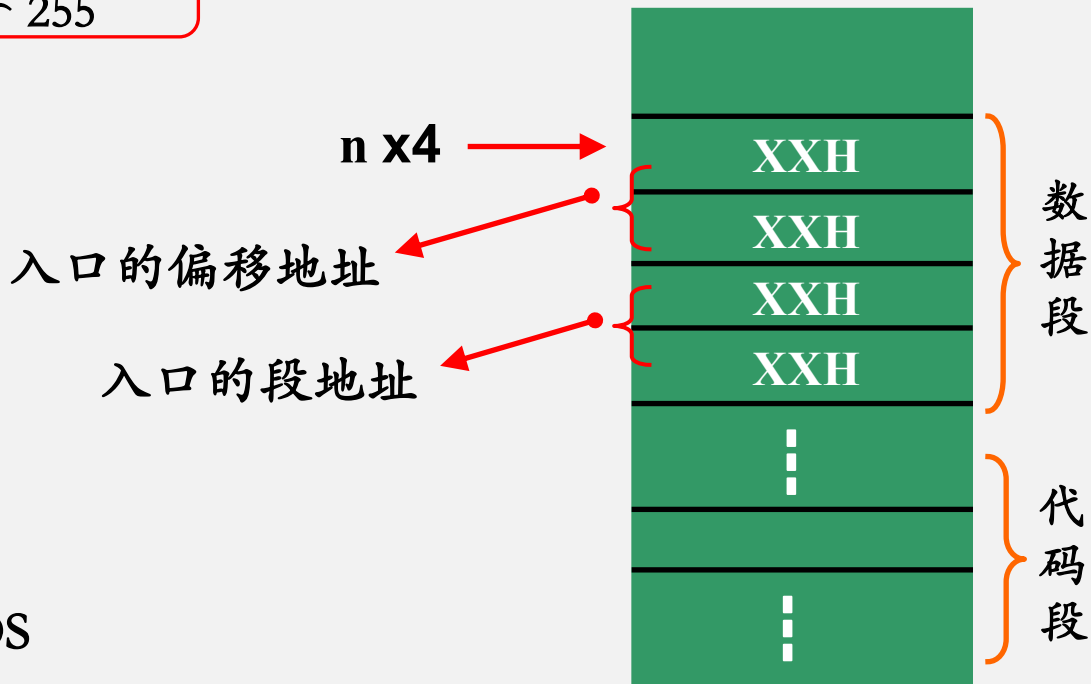
- 说明:

- nx4



存放中断服务子程序入口
地址的单元的偏移地址

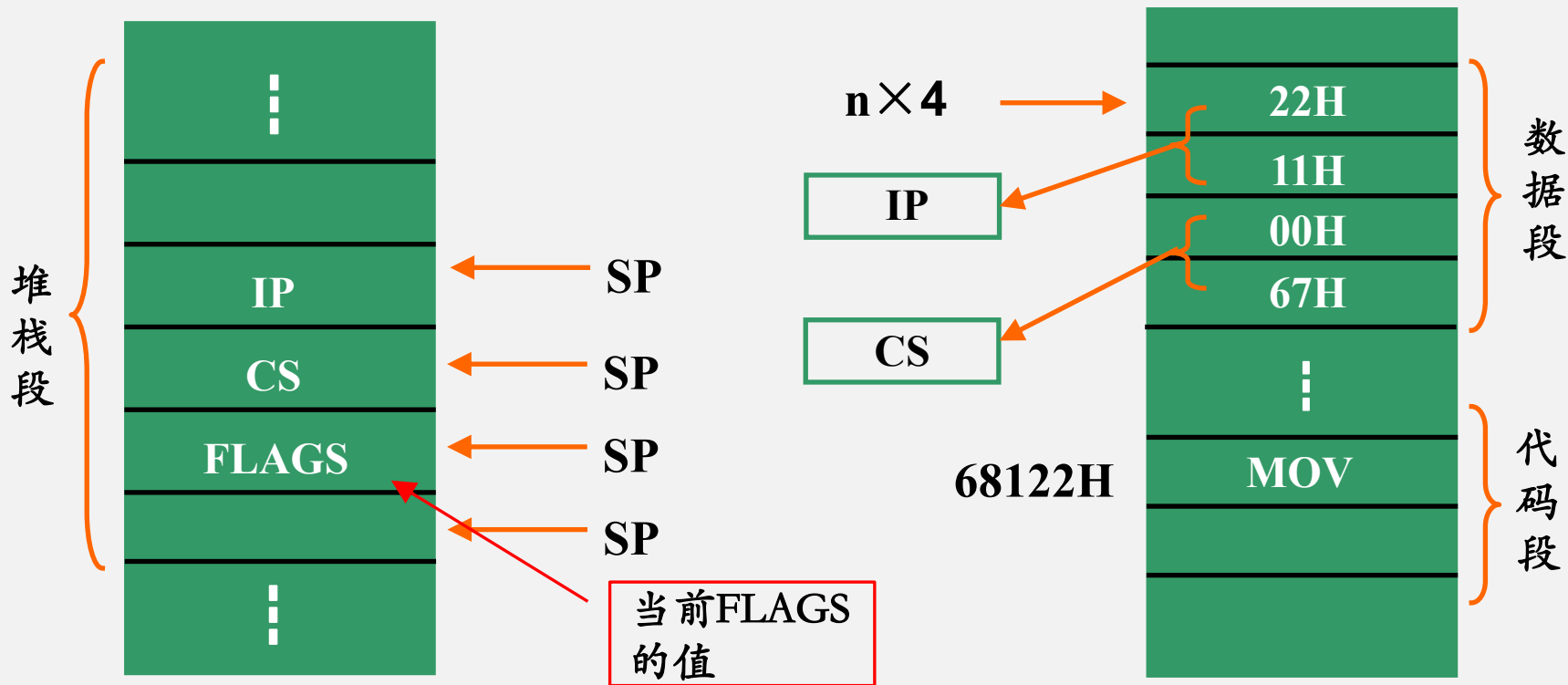
该单元在数据段, 段地址=DS



中断指令的执行过程

- ① 将FLAGS压入堆栈；
- ② 将INT指令的下一条指令的CS、IP压栈；
- ③ 由 $n \times 4$ 得到存放中断向量的地址；
- ④ 将中断向量（中断服务程序入口地址）送CS和IP寄存器；
- ⑤ 转入中断服务程序。

中断指令的执行过程



中断指令例

执行程序段：

CS IP

⋮

6200H:010DH MOV SP, 1200H

6200H:0110H INT 21H

6200H:0112H MOV AX, BX

⋮

执行INT
指令后

SP=11FAH

SP=11FCH

SP=11FEH

SP=1200H

12H

01H

00H

62H

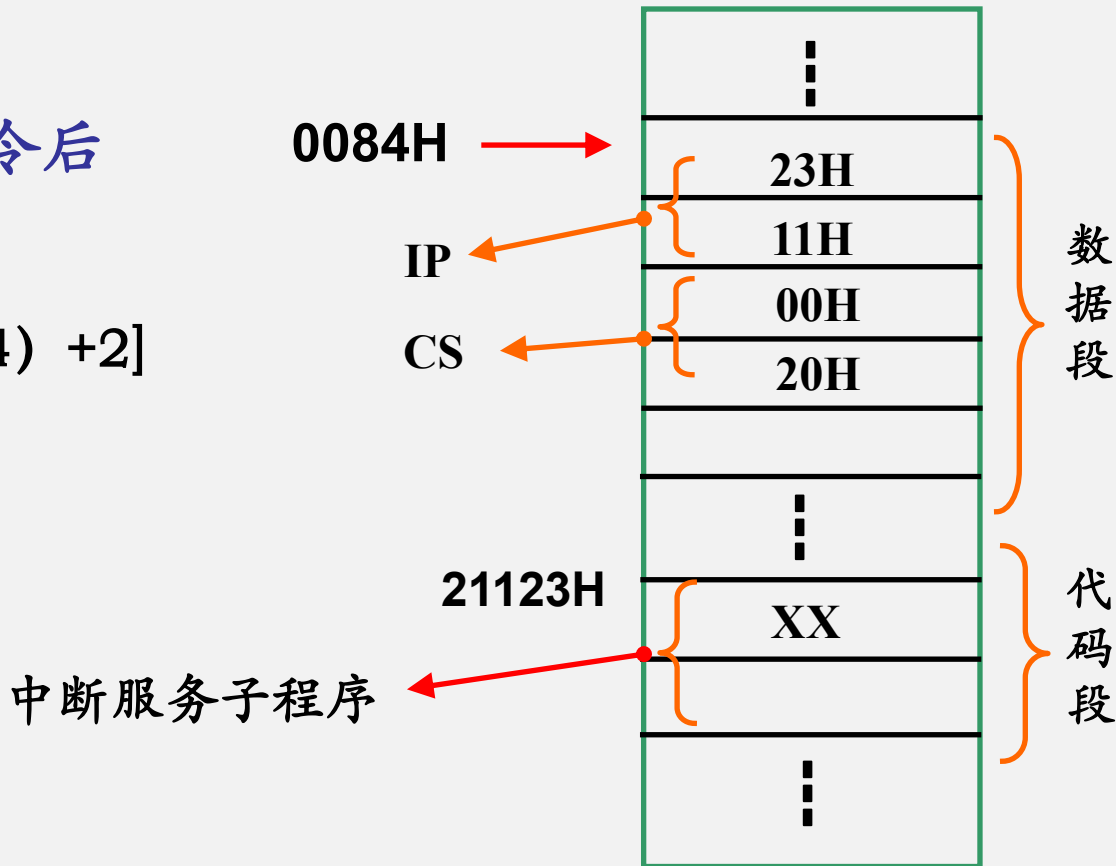
FLAGS

堆
栈
段

中断指令例

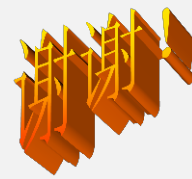
■ 执行INT 21H指令后

- $IP = [21H \times 4]$
- $CS = [(21H \times 4) + 2]$



2. 中断返回指令

- 格式：
 - IRET
- 中断服务程序的最后一条指令，负责：
 - 恢复断点
 - 恢复标志寄存器内容



处理器控制指令

处理器控制指令

- 这类指令用来对CPU进行控制，如修改标志寄存器，使CPU暂停，使CPU与外部设备同步等。

{ 对标志位的操作

{ 与外部设备的同步

处理器控制指令的控制对象是CPU

均为零操作数格式指令

标志位操作指令

■ 置标志位状态

CLC	CF←0	； 清进位标志位
STC	CF←1	； 进位标志位置位
CMC	CF←$\overline{\text{CF}}$	； 进位标志位取反
CLD	DF←0	； 清方向标志位，串操作从低地址到高地址
STD	DF←1	； 方向标志位置位，串操作从高地址到低地址
CLI	IF←0	； 清中断标志位，即关中断
STI	IF←1	； 中断标志位置位，即开中断

